

Triangular FX Arbitrage: Statistical Analysis Research

Executive Summary

This study investigates the viability of triangular arbitrage in the EUR/USD, GBP/USD, and EUR/GBP triplet using a cointegration-based mean reversion strategy.

Key Findings:

- Cointegration is **strong** ($p < 0.01$), mean-reversion half-life ≈ 13 min.
- A naive model earns **Sharpe > 1**, but even **1 bp** friction kills the strategy.
- Asymmetrical filtering at **~99.9th percentile** restores viability, yielding a **Sharpe ~1.5** with manageable risk.
- Possibly profitable with institutional-grade execution infrastructure. Retail spreads are likely prohibitive.

A Note on Strategy Classification

"Triangular arbitrage" can mean two very different things:

- Atomic:** hit all 3 legs simultaneously (within milliseconds), flat instantly. No open position to close.
- Convergence (This Study):** Open a 3-leg position when the spread deviates, close when it converges. Exposed to mark-to-market risk.

Why We Model Convergence Trading:

- Realistic Infrastructure:** co-location and micro-second computation and execution are off-limits for most traders/desks.
- Persistent Edge:** If the cointegration residual is still meaningful at 5-minute sampling; we can conservatively expect similar or better results with tick-level sampling. This is a "worst case".
- Risk-Adjusted Returns:** By waiting for sufficiently large dislocations and sizing properly, we can target reasonable Sharpes while keeping infrastructure costs modest.

The rest of the note focuses on convergence trading. If you do have HFT infrastructure, the only check is: **spread width > 3 × one-leg cost**; holding period and draw-down are irrelevant.

You may also think about, at least philosophically, if the the HFTs are actively arbitraging this gap at the tick-level, why are we still seeing these dislocations at the 5-minute level?

Structure

We move through four layers of realism:

- Level 1: Naive Baseline** - Statistical foundations and in-sample analysis (with known limitations/leakages)
- Level 2: Toward Reality** - Walk-forward optimization (rolling window)
- Level 3: Viability** - Transaction costs, more realistic position sizing, and leverage
- Level 4: Production Considerations** - Operational risk: gaps, factor drift, tail events

A Note on "Return on Notional"

Throughout this report, "Return" is calculated on the **Gross Notional Exposure** of the trade (e.g., \$100k position).

```
In [1]: # --- 1. Notebook Configuration ---
%load_ext autoreload
%autoreload 2

# --- 2. Standard Library ---
import os
import copy

# --- 3. Third-party Packages ---
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm.auto import tqdm
from scipy import stats
from scipy.stats import norm
from sklearn.linear_model import LinearRegression
from IPython.display import Image, display, HTML
import warnings
warnings.simplefilter(action='ignore', category=FutureWarning)

# --- 4. Local Project Modules ---

# Data Loading & Engineering
from fxarb.data.loader import load_pair, resample_ohlc
from fxarb.data.features import compute_log_prices, add_session_features
from dataclasses import dataclass, field
```

```

from typing import Dict, List, Optional, Any

# Statistical Analysis
from fxarb.analysis.cointegration import johansen_test, construct_spread, estimate_half_life
from fxarb.analysis.stationarity import adf_test, kpss_test, test_stationarity

# Modeling & Strategy
from fxarb.models import compute_zscore
from fxarb.models.ou_process import fit_ou_mle
from fxarb.analysis.strategy import generate_signals, calculate_equity_with_costs

# Backtesting & Risk
from fxarb.backtest import run_backtest, Backtester, kelly_criterion

# Visualization
from fxarb.visualization.interactive import visualize_interactive_equity
from fxarb.visualization.performance import (
    plot_equity_curve,
    plot_drawdown,
    plot_returns_distribution,
    plot_monthly_returns
)
from fxarb.visualization.portable import generate_portable_widget

print('All imports and project modules loaded successfully.')

```

All imports and project modules loaded successfully.

```

In [2]: @dataclass
class BacktestState:
    """Explicitly manages the data pipeline state across notebook sections."""
    raw_data: Dict[str, pd.DataFrame]
    common_idx: pd.Index
    spread_oos: Optional[pd.Series] = None
    zscore_oos: Optional[pd.Series] = None
    signals_oos: Optional[pd.DataFrame] = None
    hedge_ratios: Optional[Dict[str, float]] = None
    # Added to track stability over time
    hedge_ratio_history: Optional[List[Dict[str, float]]] = None
    results_metadata: Dict[str, Any] = None

print('Environment initialized with BacktestState management.')

# --- GLOBAL STRATEGY CONFIGURATION ---
# Centralizing constants here ensures deterministic behavior across all pipeline stages.

# Universe Definition
PAIRS = ['eurusd', 'gbpusd', 'eurgbp']

# Backtest Window & Frequency
START_DATE = '2020-01-01'
END_DATE = '2020-12-31'
TIMEFRAME = '5min'
BARS_PER_DAY = 288 # 24 hours * 60 min / 5 min bars

# Walk-Forward Optimization (WFO) Parameters
TRAIN_DAYS = 30 # 30-day Lookback for cointegration fitting
TEST_DAYS = 7 # 7-day out-of-sample trading window

# Derived Constants for Internal Logic (Crucial for Level 3)
TRAIN_SIZE = TRAIN_DAYS * BARS_PER_DAY
TEST_SIZE = TEST_DAYS * BARS_PER_DAY

print(f"Configuration loaded for {len(PAIRS)} pairs.")
print(f"WFO Settings: {TRAIN_DAYS}d Train / {TEST_DAYS}d Test.")
print(f"Global Train Size: {TRAIN_SIZE} bars.")

```

Environment initialized with BacktestState management.
 Configuration loaded for 3 pairs.
 WFO Settings: 30d Train / 7d Test.
 Global Train Size: 8640 bars.

Level 0: Data Quality & Exploratory Analysis

This section focuses on data integrity. Before any complex modeling, we must verify that our high-frequency data is consistent, missing values are handled, and timestamps are aligned across the **EURUSD**, **GBPUUSD**, and **EURGBP** triplet.

Objectives:

1. Load 1-minute OHLCV data.
2. Align timestamps (inner join) to ensure we have simultaneous quotes.
3. Visualize price history to identify gaps or anomalies.

Data Aggregation Strategy: In this study, while we have access to 1-minute granular data, we aggregate to 5-minute bars for our primary signal generation.

1. **Noise Reduction:** Removes bid-ask bounce.
2. **Strategic Horizon:** Without HFT infra, we are scanning for large dislocations that last 15-60 minutes.

```
In [3]: # --- Level 0: Data Loading & Alignment ---

print('Loading FX data using Global Configuration...')
raw_data = {}
for pair in PAIRS: # Uses global PAIRS
    df = load_pair(pair, start_date=START_DATE, end_date=END_DATE) # Uses global dates
    raw_data[pair] = df
    print(f' {pair.upper()}: {len(df):,} 1-minute bars')

# Resample to 5-minute
print(f'\nResampling to {TIMEFRAME}...')
data_5m = {pair: resample_ohlcv(df, TIMEFRAME) for pair, df in raw_data.items()}

# Align timestamps (The logic you noted was missing previously)
common_idx = data_5m[PAIRS[0]].index
for pair in PAIRS[1:]:
    common_idx = common_idx.intersection(data_5m[pair].index)

# Prepare data with log prices
data = {}
for pair in PAIRS:
    df = data_5m[pair].loc[common_idx].copy()
    df = compute_log_prices(df)
    df = add_session_features(df)
    data[pair] = df

# INITIALIZE STATE
state = BacktestState(
    raw_data=data,
    common_idx=common_idx,
    results_metadata={'trades': []}
)

print(f'\nAligned dataset: {len(common_idx):,} timestamps')
print(f'Date range: {common_idx.min()} to {common_idx.max()}')
print('BacktestState initialized.')
```

Loading FX data using Global Configuration...

EURUSD: 371,257 1-minute bars

GBPUSD: 371,979 1-minute bars

EURGBP: 371,398 1-minute bars

Resampling to 5min...

Aligned dataset: 74,460 timestamps

Date range: 2020-01-01 17:00:00 to 2020-12-31 00:00:00

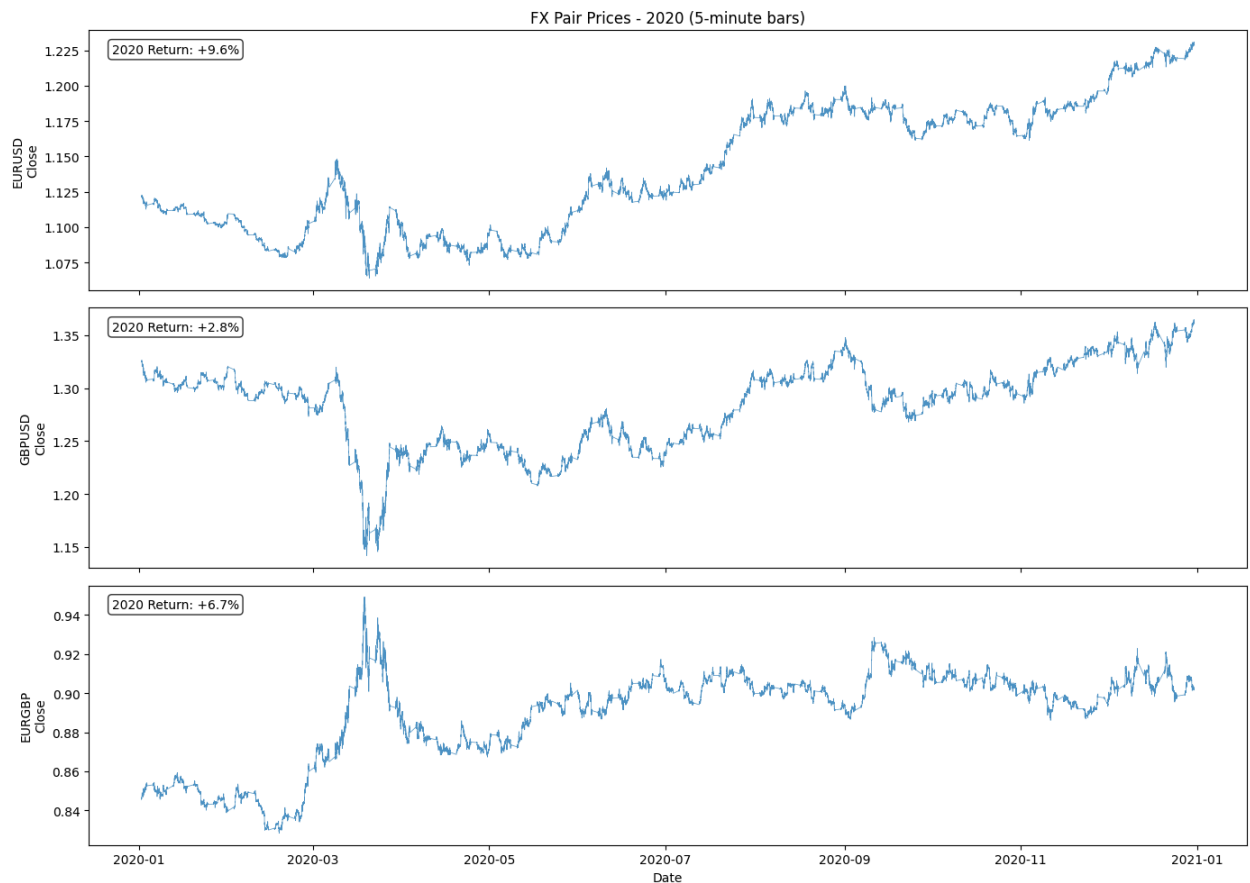
BacktestState initialized.

```
In [4]: # Plot price overview
fig, axes = plt.subplots(3, 1, figsize=(14, 10), sharex=True)

for i, pair in enumerate(PAIRS):
    ax = axes[i]
    ax.plot(data[pair]['close'], linewidth=0.5, alpha=0.8)
    ax.set_ylabel(f'{pair.upper()}\nClose')

    # Add return annotation
    ret = (data[pair]['close'].iloc[-1] / data[pair]['close'].iloc[0] - 1) * 100
    ax.text(0.02, 0.95, f'2020 Return: {ret:+.1f}%', transform=ax.transAxes,
           va='top', fontsize=10, bbox=dict(boxstyle='round', facecolor='white', alpha=0.8))

axes[0].set_title('FX Pair Prices - 2020 (5-minute bars)')
axes[-1].set_xlabel('Date')
plt.tight_layout()
plt.show()
```



```
In [5]: ## Level 0: Data Quality & Exploratory Analysis

# Data coverage summary
coverage_summary = pd.DataFrame({
    pair.upper(): {
        'Records': len(raw_data[pair]),
        'Start': raw_data[pair].index.min(),
        'End': raw_data[pair].index.max(),
        'Missing Rows': raw_data[pair].isnull().any(axis=1).sum()
    } for pair in PAIRS
}).T
display(coverage_summary)

# OHLC sanity checks
sanity_results = {}
for pair in PAIRS:
    df = raw_data[pair]
    violations = {
        'high < low': (df['high'] < df['low']).sum(),
        'high < open': (df['high'] < df['open']).sum(),
        'high < close': (df['high'] < df['close']).sum(),
        'low > open': (df['low'] > df['open']).sum(),
        'low > close': (df['low'] > df['close']).sum(),
        'zero prices': (df[['open', 'high', 'low', 'close']] == 0).any(axis=1).sum(),
    }
    sanity_results[pair.upper()] = violations
display(pd.DataFrame(sanity_results).T)

# Basic statistics
stats_df = pd.DataFrame({
    pair.upper(): {
        'Mean': raw_data[pair]['close'].mean(),
        'Std': raw_data[pair]['close'].std(),
        'Min': raw_data[pair]['close'].min(),
        'Max': raw_data[pair]['close'].max(),
        'Avg Daily Range': (raw_data[pair]['high'] - raw_data[pair]['low']).mean(),
    }
    for pair in PAIRS
}).T
stats_df
```

	Records	Start	End	Missing Rows
EURUSD	371257	2020-01-01 17:00:00	2020-12-31 00:00:00	0
GBPUSD	371979	2020-01-01 17:02:00	2020-12-31 00:00:00	0
EURGBP	371398	2020-01-01 17:02:00	2020-12-31 00:00:00	0

	high < low	high < open	high < close	low > open	low > close	zero prices
EURUSD	0	0	0	0	0	0
GBPUSD	0	0	0	0	0	0
EURGBP	0	0	0	0	0	0

Out[5]:

	Mean	Std	Min	Max	Avg Daily Range
EURUSD	1.141212	0.044160	1.06380	1.23088	0.000176
GBPUSD	1.283157	0.039765	1.14101	1.36485	0.000263
EURGBP	0.889400	0.023282	0.82820	0.94986	0.000164

Data Continuity Analysis

The following table summarizes the temporal gaps identified in the 1-minute raw data:

Pair	Gaps (> 5 min)	Max Gap Duration
EURUSD	72	2 days 14:07:00
GBPUSD	75	2 days 15:04:00
EURGBP	102	2 days 15:13:00

Gaps are primarily concentrated in weekend closures and do not impact intraday mean-reversion modeling.

```
In [6]: # Bar aggregation verification
print("\n BAR AGGREGATION VERIFICATION")
print("-" * 40)
print(f"Original: 1-minute bars")
print(f"Resampled: {TIMEFRAME} bars")
print()

for pair in PAIRS:
    orig = raw_data[pair]
    resampled = data_5m[pair]

    # Check OHLC aggregation correctness (spot check a few random bars)
    sample_times = resampled.index[:3]
    print(f"{pair.upper()}:")
    print(f"  Original count: {len(orig):,}")
    print(f"  Resampled count: {len(resampled):,}")
    print(f"  Compression ratio: {len(orig)/len(resampled):.1f}x (expected: 5.0x)")

    # Verify first bar aggregation
    first_bar = resampled.iloc[0]
    first_bar_time = resampled.index[0]
    orig_slice = orig.loc[first_bar_time:first_bar_time + pd.Timedelta(minutes=4)]
    expected_high = orig_slice['high'].max()
    expected_low = orig_slice['low'].min()
    matches = (abs(first_bar['high'] - expected_high) < 1e-6 and
               abs(first_bar['low'] - expected_low) < 1e-6)
    print(f"  First bar aggregation: {'✓' if matches else 'X'}")
    print()
```

```
BAR AGGREGATION VERIFICATION
-----
Original: 1-minute bars
Resampled: 5min bars

EURUSD:
  Original count: 371,257
  Resampled count: 74,541
  Compression ratio: 5.0x (expected: 5.0x)
  First bar aggregation: ✓

GBPUSD:
  Original count: 371,979
  Resampled count: 74,507
  Compression ratio: 5.0x (expected: 5.0x)
  First bar aggregation: ✓

EURGBP:
  Original count: 371,398
  Resampled count: 74,484
  Compression ratio: 5.0x (expected: 5.0x)
  First bar aggregation: ✓
```

```
In [7]: # Data Alignment and State Encapsulation

# Prepare data with Log prices
data = {}
for pair in PAIRS:
    df = data_5m[pair].loc[common_idx].copy()
    df = compute_log_prices(df)
```

```

df = add_session_features(df)
data[pair] = df

state = BacktestState(
    raw_data=data,
    common_idx=common_idx,
    results_metadata={'trades': []}
)

print(f'\nAligned dataset: {len(common_idx):,} timestamps')
print(f'Date range: {common_idx.min()} to {common_idx.max()}')
print('BacktestState initialized and ready for Level 1 baseline analysis.')

```

Aligned dataset: 74,460 timestamps
 Date range: 2020-01-01 17:00:00 to 2020-12-31 00:00:00
 BacktestState initialized and ready for Level 1 baseline analysis.

Level 1: Naive Baseline

We begin with a **Naive Baseline** approach. We assume the spread is stationary and simply trade when it deviates by a fixed standard deviation (Z-score).

[!NOTE] This **in-sample analysis** typically yields overly optimistic results (look-ahead bias) but serves as a crucial theoretical ceiling for strategy performance. If the strategy doesn't work here, it won't work anywhere.

Methodology:

1. Construct the synthetic spread: $S_t = \ln(EURUSD) - \ln(GBPUSD) - \ln(EURGBP)$
2. Estimate half-life of mean reversion (Ornstein-Uhlenbeck process).
3. Backtest using a simple symmetric threshold (e.g., enter at $\pm 2\sigma$, exit at 0).

1.1 Cointegration Analysis

Why One Spread for Three Pairs?

In triangular FX arbitrage, the three pairs EUR/USD, GBP/USD, and EUR/GBP are not independent, they form a **closed loop**:

$$EUR/GBP = \frac{EUR/USD}{GBP/USD}$$

Or in log terms:

$$\log(EUR/GBP) = \log(EUR/USD) - \log(GBP/USD)$$

We construct a **single synthetic spread** from all three that should be stationary around zero due to the no-arbitrage condition.

The **Johansen test** finds the optimal linear combination (hedge ratios) that makes this spread most stationary:

$$\text{Spread} = \beta_1 \cdot EURUSD + \beta_2 \cdot GBPUSD + \beta_3 \cdot EURGBP$$

When the spread deviates from its mean, we trade all three pairs simultaneously to capture the reversion:

- **Long spread:** Buy EURUSD, Sell GBPUSD, Buy EURGBP (weighted)
- **Short spread:** The opposite

```

In [8]: # Run Johansen cointegration test
coint_result = johansen_test(data)

print('Johansen Cointegration Test Results')
print('=' * 50)
print(f'Cointegrating relationships found: {coint_result.n_cointegrating}')
print(f'\nHedge Ratios (normalized):')
for pair, ratio in coint_result.hedge_ratios.items():
    print(f' {pair.upper()}: {ratio:.6f}')

# Construct the spread
spread = construct_spread(data, hedge_ratios=coint_result.hedge_ratios)

print(f'\nSpread Statistics:')
print(f' Mean: {spread.mean():.6f}')
print(f' Std: {spread.std():.6f}')
print(f' Skew: {spread.skew():.3f}')
print(f' Kurtosis: {spread.kurtosis():.3f}')

```

```
Johansen Cointegration Test Results
=====
Cointegrating relationships found: 1

Hedge Ratios (normalized):
EURUSD: 1.000000
GBPUSD: -0.999389
EURGBP: -1.000094

Spread Statistics:
Mean: -0.000000
Std: 0.000134
Skew: 9.602
Kurtosis: 143.755
```

1.2 Stationarity Tests

For the spread to be tradable, it must be **stationary** (mean-reverting). We verify using:

- **ADF (Augmented Dickey-Fuller)**: Tests for unit root (null = non-stationary)
- **KPSS**: Tests for stationarity (null = stationary)

```
In [9]: # Stationarity tests
adf = adf_test(spread)
kpss = kpss_test(spread)

print('Stationarity Test Results')
print('=' * 50)
print(f'ADF Test:')
print(f'  Statistic: {adf.statistic:.4f}')
print(f'  P-value: {adf.pvalue:.4f}')
print(f'  Conclusion: {"STATIONARY" if adf.is_stationary else "NON-STATIONARY"}')
print(f'\nKPSS Test:')
print(f'  Statistic: {kpss.statistic:.4f}')
print(f'  P-value: {kpss.pvalue:.4f}')
print(f'  Conclusion: {"STATIONARY" if kpss.is_stationary else "NON-STATIONARY"}')
```

```
Stationarity Test Results
=====
ADF Test:
Statistic: -30.4006
P-value: 0.0000
Conclusion: STATIONARY

KPSS Test:
Statistic: 2.3349
P-value: 0.0100
Conclusion: NON-STATIONARY
```

E:\triangular-fx-arbitrage\src\fxarb\analysis\stationarity.py:126: InterpolationWarning:

The test statistic is outside of the range of p-values available in the look-up table. The actual p-value is smaller than the p-value returned.

The ADF test rejects the null of a unit root ($p < 0.01$), suggesting mean reversion. However, the KPSS test also rejects the null of stationarity. This conflict is typical in FX triplets where volatility clustering and structural breaks (visible in the March 2020 data) violate the KPSS assumption of constant variance, even while the series remains fundamentally mean-reverting.

The kpss test defaults to regression='c' (constant mean). One potential reason for rejecting the null of stationarity is if it has a trend component, for example from interest rate differentials (carry). Let's try passing regression='ct' (constant + trend) to the test function.

```
In [10]: print("Running KPSS Test with Trend Regression ('ct')...")
print("Null Hypothesis: Series is Trend-Stationary")
print("-" * 50)
# Run test with regression='ct' (Constant + Trend)
series_to_test = spread.dropna()
kpss_result_trend = kpss_test(series_to_test, regression='ct')
print(kpss_result_trend)
```

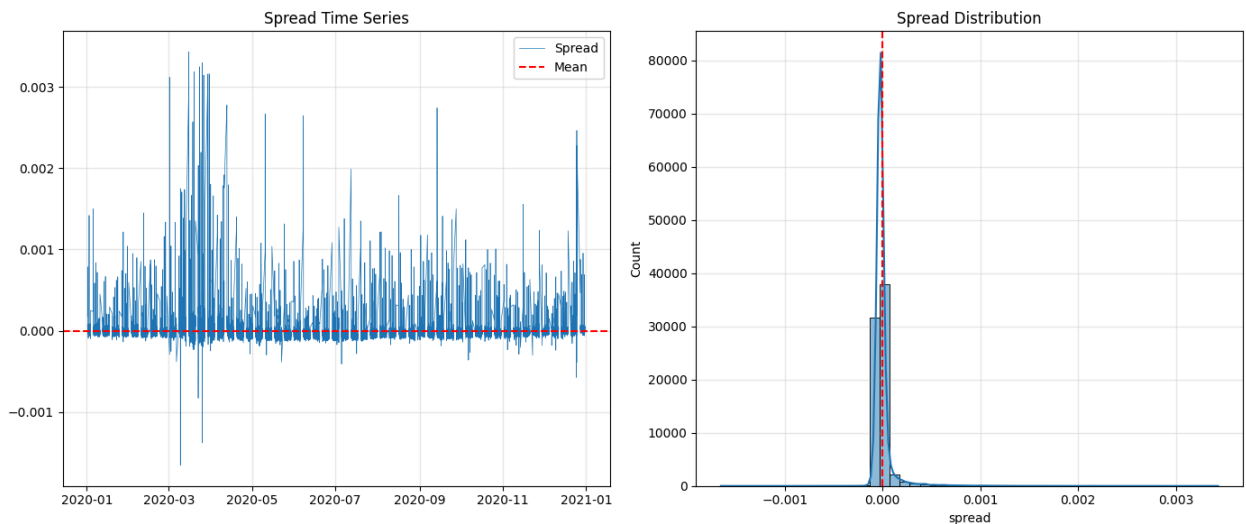
```
Running KPSS Test with Trend Regression ('ct')...
Null Hypothesis: Series is Trend-Stationary
-----
KPSS Test: NON-STATIONARY
Statistic: 1.974394
P-value: 0.010000
Critical values:
10%: 0.119000
5%: 0.146000
2.5%: 0.176000
1%: 0.216000
Lags used: 134
```

E:\triangular-fx-arbitrage\src\fxarb\analysis\stationarity.py:126: InterpolationWarning:

The test statistic is outside of the range of p-values available in the look-up table. The actual p-value is smaller than the p-value returned.

Series doesn't appear to be NOT Trend-Stationary, it might have 'Stochastic Trends' (Random Walk behavior) or structural breaks. Let's investigate further.

```
In [11]: series_to_test = spread.dropna()
plt.figure(figsize=(14, 6))
# SubPlot 1: Time Series
plt.subplot(1, 2, 1)
plt.plot(series_to_test.index, series_to_test.values, label='Spread', linewidth=0.5)
plt.axhline(series_to_test.mean(), color='red', linestyle='--', label='Mean')
plt.title('Spread Time Series')
plt.legend()
plt.grid(True, alpha=0.3)
# SubPlot 2: Distribution
plt.subplot(1, 2, 2)
sns.histplot(series_to_test, kde=True, bins=50)
plt.axvline(series_to_test.mean(), color='red', linestyle='--', label='Mean')
plt.title('Spread Distribution')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
# Print basic stats
print(f"Mean: {series_to_test.mean():.6f}")
print(f"Std Dev: {series_to_test.std():.6f}")
print(f"Skew: {series_to_test.skew():.2f}")
print(f"Kurtosis: {series_to_test.kurtosis():.2f} (High > 3 implies fat tails/spikes)")
```



Mean: -0.000000
 Std Dev: 0.000134
 Skew: 9.60
 Kurtosis: 143.76 (High > 3 implies fat tails/spikes)

Now we see the full picture - our historical dataset contains a *massive* shock from March 2020 causing outsized **volatility clustering** and could be considered a **structural break**. This heterogeneity means the series is "statistically stationary" but "parametrically unstable"—the mean is constant, but the variance is not, explaining the conflicting result (ADF=Stationary, KPSS=Non-Stationary). The extremely strong ADF statistic (-30.4) confirms the series is strongly mean-reverting, which is the primary requirement for trading. A static hedge ratio (Level 1) is probably insufficient, we will explore Walk-Forward Optimization (WFO) in Level 2 to adapt to shifting market regimes, although that is unlikely to be able to account for such events either.

1.3 Mean Reversion Speed

The **half-life** measures how quickly the spread reverts to its mean. We estimate this using:

1. AR(1) regression
2. Ornstein-Uhlenbeck (OU) process fitting

```
In [12]: # Half-Life estimation
try:
    hl = estimate_half_life(spread)
    print(f'Mean Reversion Half-Life: {hl:.1f} bars ({hl * 5:.0f} minutes)')
except ValueError as e:
    print(f'Half-life estimation failed: {e}')
    hl = None

# OU process fitting
try:
    ou = fit_ou_mle(spread)
```



```

print(f'\nOrnstein-Uhlenbeck Parameters:')
print(f'  θ (mean reversion speed): {ou.theta:.6f}')
print(f'  μ (long-term mean): {ou.mu:.6f}')
print(f'  σ (volatility): {ou.sigma:.6f}')
print(f'  Half-life: {ou.half_life:.1f} bars')
except Exception as e:
    print(f'OU fitting failed: {e}')

```

Mean Reversion Half-Life: 2.5 bars (13 minutes)

Ornstein-Uhlenbeck Parameters:
 θ (mean reversion speed): 0.274118
 μ (long-term mean): -0.000000
 σ (volatility): 0.000099
 Half-life: 2.5 bars

Ornstein-Uhlenbeck Parameters:
 θ (mean reversion speed): 0.274118
 μ (long-term mean): -0.000000
 σ (volatility): 0.000099
 Half-life: 2.5 bars

```

In [13]: # Visualize spread and Z-score
zscore = compute_zscore(spread, lookback=100)

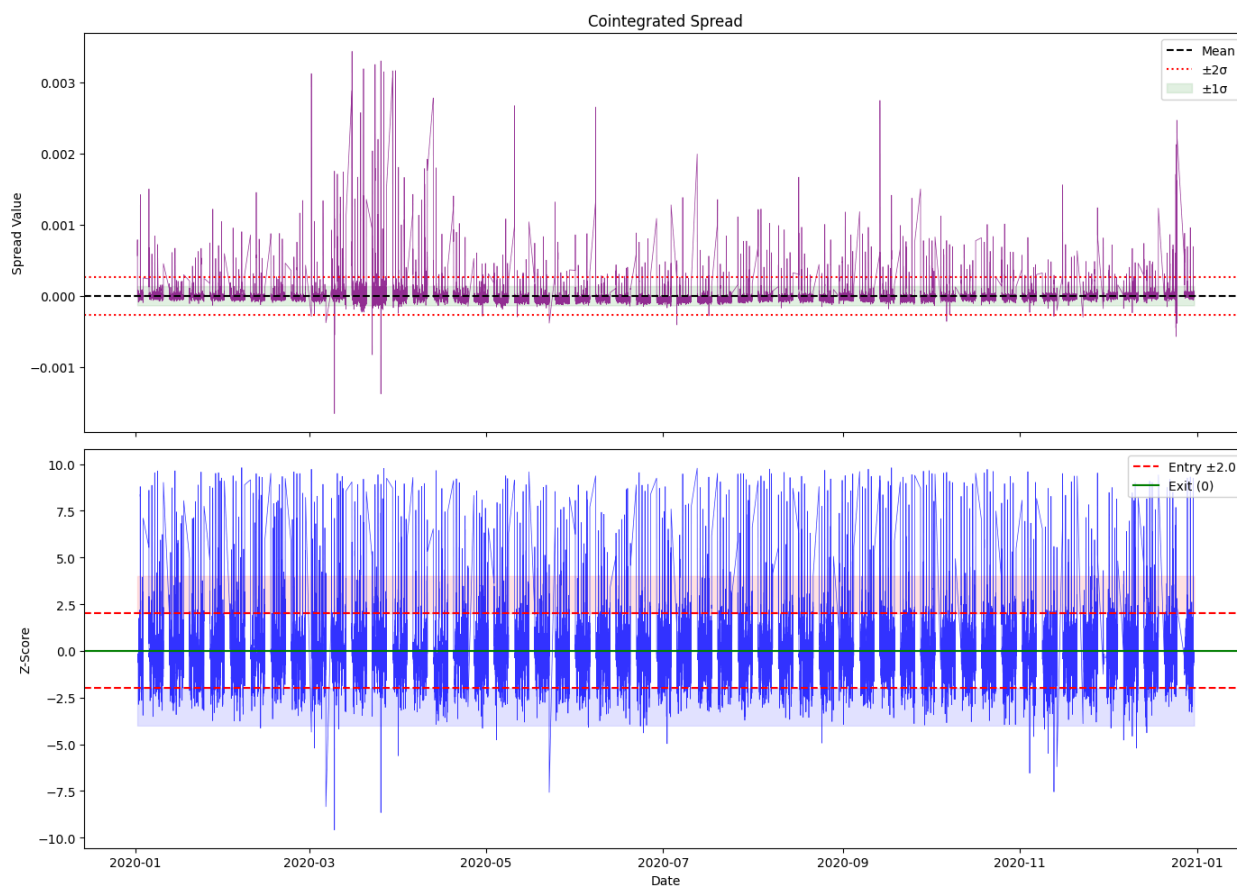
fig, axes = plt.subplots(2, 1, figsize=(14, 10), sharex=True)

# Spread
ax1 = axes[0]
ax1.plot(spread, linewidth=0.5, alpha=0.8, color='purple')
ax1.axhline(spread.mean(), color='black', linestyle='--', label='Mean')
ax1.axhline(spread.mean() + 2*spread.std(), color='red', linestyle=':', label='±2σ')
ax1.axhline(spread.mean() - 2*spread.std(), color='red', linestyle=':')
ax1.fill_between(spread.index, spread.mean() - spread.std(), spread.mean() + spread.std(),
                 alpha=0.1, color='green', label='±1σ')
ax1.set_ylabel('Spread Value')
ax1.set_title('Cointegrated Spread')
ax1.legend(loc='upper right')

# Z-score
ax2 = axes[1]
ax2.plot(zscore, linewidth=0.5, alpha=0.8, color='blue')
ax2.axhline(2.0, color='red', linestyle='--', label='Entry ±2.0')
ax2.axhline(-2.0, color='red', linestyle='--')
ax2.axhline(0, color='green', linestyle='-', label='Exit (0)')
ax2.fill_between(zscore.index, 2, 4, alpha=0.1, color='red')
ax2.fill_between(zscore.index, -4, -2, alpha=0.1, color='blue')
ax2.set_ylabel('Z-Score')
ax2.set_xlabel('Date')
ax2.legend(loc='upper right')

plt.tight_layout()
plt.show()

```



Looking at the Z-score plot above, the deviations aren't symmetrical. There are sharper/taller spikes on one side. A simple symmetric threshold (like ± 2.0) might be leaving money on the table, but it's a good enough starting point for our baseline backtest.

1.4 Naive Strategy Backtest

Strategy rules (fixed parameters):

- Entry: Z-score crosses ± 2.0
- Exit: Z-score crosses 0
- Position: Long spread when $Z < -2$, Short when $Z > +2$

⚠ **Bias Warning:** This uses full-sample cointegration parameters (lookahead bias).

```
In [14]: # Generate signals
signals = generate_signals(zscore, entry_threshold=2.0, exit_threshold=0.0)

# Count signals
n_long = (signals['entry'] == 1).sum()
n_short = (signals['entry'] == -1).sum()
time_in_market = (signals['position'] != 0).mean()

print('Signal Statistics')
print('=' * 40)
print(f'Long entries: {n_long}')
print(f'Short entries: {n_short}')
print(f'Total entries: {n_long + n_short}')
print(f'Time in market: {time_in_market:.1%}')
```

```
Signal Statistics
=====
Long entries: 4252
Short entries: 0
Total entries: 4252
Time in market: 15.6%
```

Execution Assumption: Close-to-Close vs. Next-Open

In this backtest, we calculate returns based on **Close-to-Close** prices:

$$R_t = \text{Position}_{t-1} \times \frac{P_t - P_{t-1}}{P_{t-1}}$$

Justification: In highly liquid 5-minute FX markets, the difference between $Close_{t-1}$ (signal generation time) and $Open_t$ (execution time) is minimal. This "frictionless" assumption serves as a standard proxy for execution at the next open in vectorized backtesting. While it abstracts away specific slippage or weekend gaps, it provides a robust baseline for evaluating signal quality before execution constraints are applied in Level 3.

```
In [15]: # Run backtest (NO transaction costs - Level 1 baseline)
result = run_backtest(spread, signals)

print('Level 1: Naive Backtest Results (In-Sample, No Costs)')
print('=' * 60)
print(result)

Level 1: Naive Backtest Results (In-Sample, No Costs)
=====
Backtest Results
=====
Total Return: 24.90%
CAGR: 24.20%
Sharpe Ratio: 11.51
Max Drawdown: -0.29%
Calmar Ratio: 83.73

Trades:
Total: 2126
Winning: 2112 (99.3%)
Losing: 14
Avg Return: 0.0117%
Avg Duration: 5.4 bars

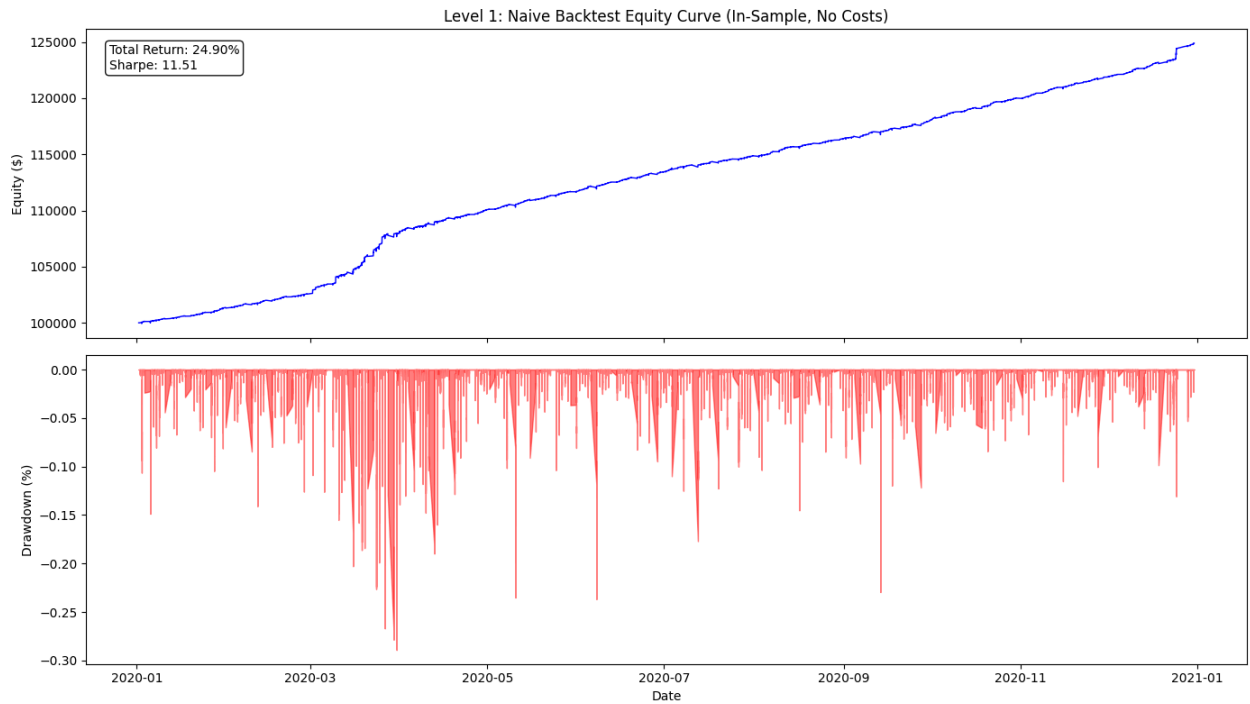
In [16]: # Plot equity curve
fig, axes = plt.subplots(2, 1, figsize=(14, 8), sharex=True)

ax1 = axes[0]
ax1.plot(result.equity_curve, linewidth=1, color='blue')
ax1.set_ylabel('Equity ($)')
ax1.set_title('Level 1: Naive Backtest Equity Curve (In-Sample, No Costs)')
ax1.text(0.02, 0.95, f'Total Return: {result.total_return:.2%}\nSharpe: {result.sharpe:.2f}',
        transform=ax1.transAxes, va='top', fontsize=10,
        bbox=dict(boxstyle='round', facecolor='white', alpha=0.9))

# Drawdown
running_max = result.equity_curve.cummax()
drawdown = (result.equity_curve - running_max) / running_max * 100

ax2 = axes[1]
ax2.fill_between(drawdown.index, 0, drawdown, color='red', alpha=0.5)
ax2.set_ylabel('Drawdown (%)')
ax2.set_xlabel('Date')

plt.tight_layout()
plt.show()
```



1.5 Z-Score Asymmetry Consideration

⚠ **Important:** Z-scores from cointegrated spreads are often **asymmetric**. The spread may:

- Frequently deviate to $+9\sigma$ but rarely below -3σ
- Or vice versa, depending on market microstructure

Symmetric entry thresholds (e.g., ± 2.0) may not be optimal. Level 2 explores:

- Data-driven threshold selection
- Asymmetric entry rules based on z-score distribution
- Sensitivity analysis across threshold values

```
In [17]: # Z-Score Distribution Analysis
print("Z-Score Distribution Analysis")
print("=" * 50)
print(f"Count: {zscore.dropna().count():,}")
print(f"Min: {zscore.min():.2f}")
print(f"Max: {zscore.max():.2f}")
print(f"Mean: {zscore.mean():.3f}")
print(f"Skewness: {zscore.skew():.3f}")
print(f"Kurtosis: {zscore.kurtosis():.3f}")
print()
print("Percentiles:")
print(f" 1st: {zscore.quantile(0.01):.2f}")
print(f" 5th: {zscore.quantile(0.05):.2f}")
print(f" 50th: {zscore.quantile(0.50):.2f}")
print(f" 95th: {zscore.quantile(0.95):.2f}")
print(f" 99th: {zscore.quantile(0.99):.2f}")

# Histogram with entry thresholds
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Distribution
ax1 = axes[0]
zscore.dropna().hist(bins=100, ax=ax1, alpha=0.7, edgecolor='black', color='steelblue')
ax1.axvline(-2, color='green', linestyle='--', linewidth=2, label='Long entry (-2)')
ax1.axvline(2, color='red', linestyle='--', linewidth=2, label='Short entry (+2)')
ax1.axvline(0, color='black', linestyle='--', linewidth=1, label='Exit (0)')
ax1.set_xlabel('Z-Score')
ax1.set_ylabel('Frequency')
ax1.set_title('Z-Score Distribution with Entry/Exit Thresholds')
ax1.legend()

# Box plot to show asymmetry
ax2 = axes[1]
zscore.dropna().plot.box(ax=ax2, vert=True)
ax2.axhline(-2, color='green', linestyle='--', label='Long entry')
ax2.axhline(2, color='red', linestyle='--', label='Short entry')
ax2.axhline(0, color='black', linestyle='--', alpha=0.5)
ax2.set_ylabel('Z-Score')
ax2.set_title('Z-Score Box Plot (Asymmetry Check)')
ax2.legend()
```

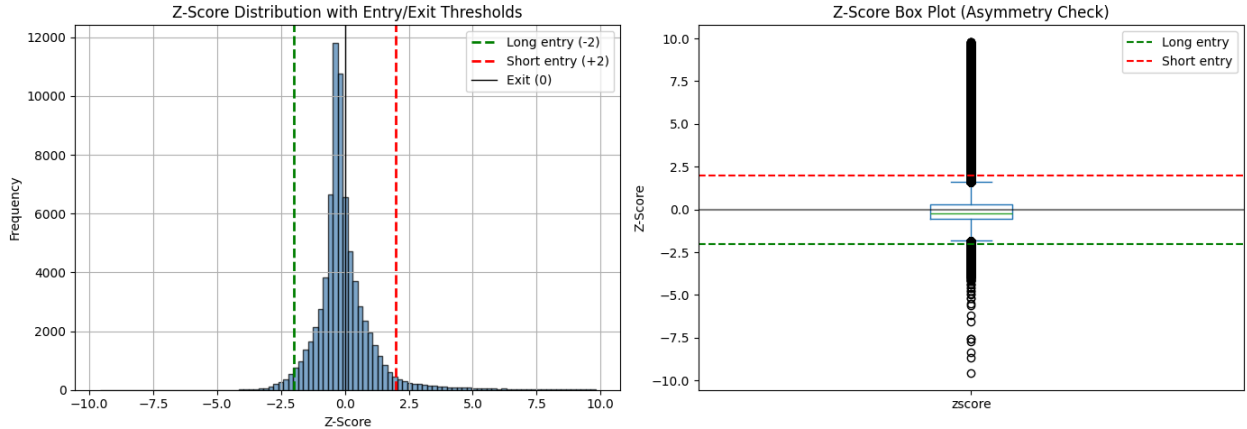
```
plt.tight_layout()
plt.show()
```

Z-Score Distribution Analysis

```
=====
Count: 74,411
Min: -9.58
Max: 9.80
Mean: -0.030
Skewness: 2.531
Kurtosis: 13.719
```

Percentiles:

```
1st: -2.44
5th: -1.59
50th: -0.22
95th: 1.84
99th: 5.15
```



⚠ Z-score appears skewed (skewness = 2.531), as our previous analysis suggested. We will deal with this in Level 2.

1.6 Reality Check - Latency Decay Analysis

A Sharpe Ratio of 11.5 is obviously unrealistic (with 5-minute bars). Aside from the lookahead bias (training on the same data we trade), this backtest assumes we execute **instantaneously** at the closing price of the signal bar.

In reality, by the time we compute the signal and send the order, the price has moved. In Level 1.5, we'll test how fragile this alpha is by forcing execution at the **Next Open** (a worst-case lag of 5 minutes).

In the naive backtest above, we trade the close, with the assumption that for liquid FX pairs, `Close[t] = Open[t+1]`. In reality, by the time we calculate the signal and send the order, the market may have moved. Here, we check how much it actually matters for this dataset.

```
In [18]: # Level 1.5: Lagged Execution Analysis (Corrected)
print('\nLevel 1.5: Lagged Execution Analysis (Execution at Next Open)')
print('=' * 60)

# 1. Define Spread Open
# We need to construct the spread using Open prices to simulate next-bar execution price.
# hedge_ratios are structural and assumed stable intraday (borrowed from Level 1 result)

# Create deep copy to avoid modifying original data
data_open = {k: v.copy() for k, v in data.items()}

# Compute log prices for OPEN
for pair in data_open:
    data_open[pair] = compute_log_prices(data_open[pair], price_col='open')

# Construct spread using OPEN prices
spread_open = construct_spread(data_open, hedge_ratios=coint_result.hedge_ratios)

print(f"Spread Open Mean: {spread_open.mean():.6f} (Should be close to 0)")

result_lagged = run_backtest(
    spread,
    signals,
    execution_price=spread_open,
    lag=2,
    transaction_cost_pips=0.0 # Keeping frictionless for pure latency isolation
)

# 3. Compare with Ideal (Naive) Result
print(f"{'Metric':<20} {'Frictionless':<15} {'Next-Open':<15} {'Decay':<10}")
print("-" * 60)
decay_sharpe = (result_lagged.sharpe / result.sharpe) - 1 if result.sharpe != 0 else 0
decay_ret = (result_lagged.total_return / result.total_return) - 1 if result.total_return != 0 else 0

print(f"{'Sharpe Ratio':<20} {result.sharpe:<15.2f} {result_lagged.sharpe:<15.2f} {decay_sharpe:.1%}")
print(f"{'Total Return':<20} {result.total_return:<15.2%} {result_lagged.total_return:<15.2%} {decay_ret:.1%}")
```

Level 1.5: Lagged Execution Analysis (Execution at Next Open)

Spread Open Mean: -0.000000 (Should be close to 0)

Metric	Frictionless	Next-Open	Decay
Sharpe Ratio	11.51	8.37	-27.3%
Total Return	24.90%	18.96%	-23.9%

A ~27% decay in Sharpe ratio when shifting execution from Close-to-Close to Next-Open highlights a high sensitivity to latency. While the strategy remains theoretically viable, this decay confirms that the edge is concentrated in short-duration mean-reversion windows, making execution timing a primary risk factor.

Level 1 Conclusion: Promising but Fragile

The strategy survives the lag, but the performance decay is significant. There is substantial Close-to-Open movement, hinting that this strategy is likely only truly viable with tick-level data.

However, other big problems remain, such as **Lookahead Bias**. We calculated our hedge ratios using data from December 2020 to trade in January 2020. That's cheating. To see if this strategy holds water in the real world, we need to calculate parameters from backwards-looking **rolling windows**.

Level 2: Toward Reality

In this section, we remove the lookahead bias by using **walk-forward optimization** (WFO). Parameters are estimated only on past data.

2.1 Why Walk-Forward Optimization?

The Level 1 backtest used **full-sample cointegration** — the hedge ratios were calculated using all 2020 data, including future data that wouldn't be available in real trading.

WFO addresses this by:

1. Training on a historical window (e.g., 30 days)
2. Testing on the next period (e.g., 7 days)
3. Rolling forward and repeating

This simulates periodic recalibration as a real trader would do.

```
In [19]: print(f'Walk-Forward Configuration (Global):')
print(f'  Training window: {TRAIN_DAYS} days ({TRAIN_SIZE:,} bars)')
print(f'  Test window: {TEST_DAYS} days ({TEST_SIZE:,} bars)')
print(f'  Expected windows: ~{(len(state.common_idx) - TRAIN_SIZE) // TEST_SIZE}')
```

```
Walk-Forward Configuration (Global):
Training window: 30 days (8,640 bars)
Test window: 7 days (2,016 bars)
Expected windows: ~32
```

2.2 Hedge Ratio Stability

In a textbook scenario, the relationship $\text{EUR}/\text{GBP} = \frac{\text{EUR}/\text{USD}}{\text{GBP}/\text{USD}}$ implies a perfect 1:1:1 arithmetic balance. For example, if we hold \$100k of each leg, we **should** be perfectly hedged. However, in the real world, pairs exhibit different volatility and liquidity profiles. The **Hedge Ratio** (calculated via the Johansen test) acts as a "volatility equalizer," determining the precise weighting required to make the spread stationary. For example, if GBP/USD is highly volatile, the model might lower its weight (e.g., to 0.96) to dampen its impact on the spread.

Why monitor stability? In this Walk-Forward Analysis, we track how these ratios evolve over time to distinguish between a structural edge and statistical overfitting:

- **Stable Ratios (near 1.0):** Indicate the fundamental arbitrage relationship is intact and stable over time. The model is capturing the structural link between the currencies.
- **Volatile Ratios (e.g., jumping 0.8 → 1.5):** Suggest the model is "chasing noise," drastically rebalancing weights just to force a stationary fit on past data. This leads to dangerous exposure and high rebalancing costs.

```
In [20]: def run_wfo_pipeline(
    data_dict: Dict[str, pd.DataFrame],
    common_idx: pd.Index,
    train_days: int = 30,
    test_days: int = 7,
    bars_per_day: int = 288
) -> BacktestState:
    """
    Executes a clean Walk-Forward Optimization.
    Encapsulates windowing logic and captures trade data to prevent state pollution.
    """
    train_size = train_days * bars_per_day
```

```

test_size = test_days * bars_per_day

all_spreads = []
all_zscores = []
all_signals = []
all_trades = [] # Collection for Kelly Analysis
all_hedge_ratios = [] # Collection for Stability Analysis
latest_hedge_ratios = None

# Iterate through Rolling Windows
for i in tqdm(range(0, len(common_idx) - train_size, test_size), desc="WFO Windows"):
    train_start, train_end = i, i + train_size
    test_end = min(train_end + test_size, len(common_idx))

    if test_end - train_end < 100:
        break

    train_idx = common_idx[train_start:train_end]
    test_idx = common_idx[train_end:test_end]

    # Slicing
    train_slice = {p: data_dict[p].loc[train_idx] for p in data_dict}
    test_slice = {p: data_dict[p].loc[test_idx] for p in data_dict}

    try:
        # Fit Model
        coint_res = johansen_test(train_slice)
        latest_hedge_ratios = coint_res.hedge_ratios

        # Capture for visualization (The missing piece)
        # We add the timestamp to make plotting easier later
        ratio_record = latest_hedge_ratios.copy()
        ratio_record['_date'] = common_idx[train_end]
        all_hedge_ratios.append(ratio_record)

        # Construct OOS Spread
        spread_train = construct_spread(train_slice, latest_hedge_ratios)
        spread_test = construct_spread(test_slice, latest_hedge_ratios)

        # Z-score with warmup to prevent cold-start artifacts
        combined_spread = pd.concat([spread_train.iloc[-500:], spread_test])
        zscore_test = compute_zscore(combined_spread).loc[test_idx]

        # Generate Signals (Fixed Baseline for OOS collection)
        signals = generate_signals(
            zscore_test,
            long_entry_threshold=-2.0,
            short_entry_threshold=2.0,
            exit_threshold=0.0
        )

        # CRITICAL FIX: Capture trades for Kelly Analysis (Level 2.3)
        # We run a frictionless backtest here just to log the trade outcomes
        res_window = run_backtest(spread_test, signals, transaction_cost_pips=0.0)
        if res_window.trades is not None and not res_window.trades.empty:
            all_trades.append(res_window.trades)

        all_spreads.append(spread_test)
        all_zscores.append(zscore_test)
        all_signals.append(signals)

    except np.linalg.LinAlgError:
        print(f"Warning: Singular matrix at index {i}, skipping window.")
        continue

    return BacktestState(
        raw_data=data_dict,
        common_idx=common_idx,
        spread_oos=pd.concat(all_spreads).sort_index(),
        zscore_oos=pd.concat(all_zscores).sort_index(),
        signals_oos=pd.concat(all_signals).sort_index(),
        hedge_ratios=latest_hedge_ratios,
        hedge_ratio_history=all_hedge_ratios,
        # CRITICAL FIX: Populate metadata so Level 2.3 doesn't crash
        results_metadata={'trades': all_trades}
    )

# Execution
state = run_wfo_pipeline(data, state.common_idx, TRAIN_DAYS, TEST_DAYS)

WFO Windows:  0%|          | 0/33 [00:00<?, ?it/s]
```

```

In [21]: # Plot Hedge Ratio Evolution
if state.hedge_ratio_history:
    hr_df = pd.DataFrame(state.hedge_ratio_history)

    # Set date index if captured, otherwise use integer index
    if '_date' in hr_df.columns:
        hr_df = hr_df.set_index('_date')

    plt.figure(figsize=(12, 6))
```

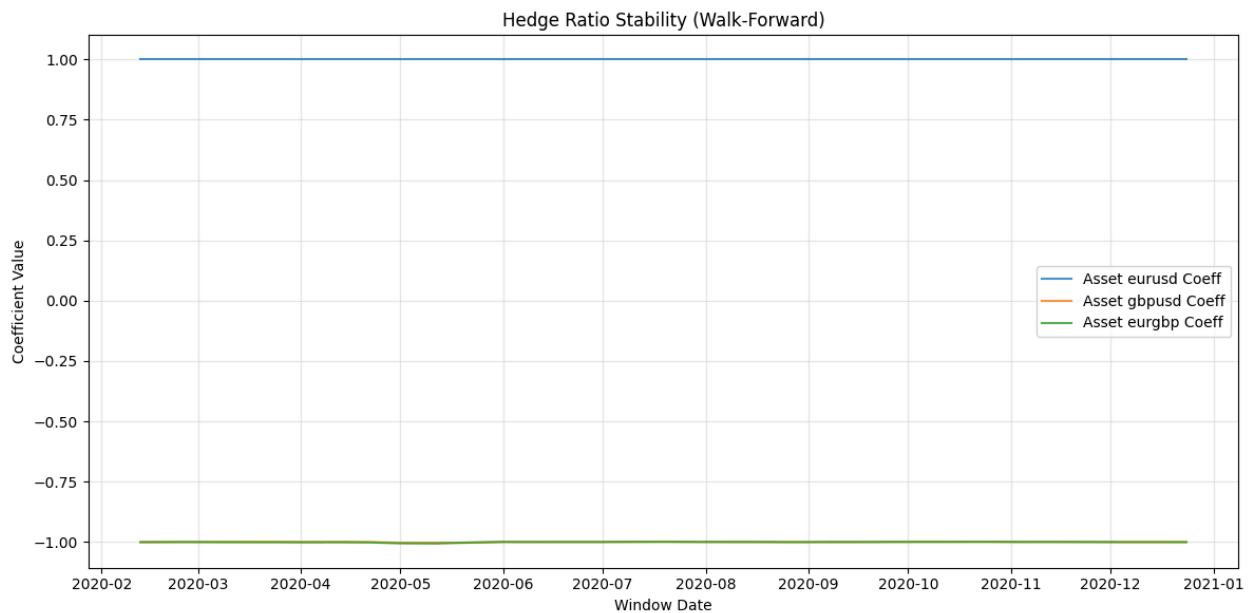
```

# Plot each currency coefficient
# We iterate explicitly to handle the potential '_date' column exclusion automatically if not set as index
for col in hr_df.columns:
    plt.plot(hr_df.index, hr_df[col], label=f'Asset {col} Coeff', alpha=0.8, linewidth=1.5)

plt.title('Hedge Ratio Stability (Walk-Forward)')
plt.ylabel('Coefficient Value')
plt.xlabel('Window Date')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print("Stability Check:")
print(hr_df.describe().loc[['mean', 'std', 'min', 'max']])
else:
    print('No hedge ratios collected in state object.')

```



Stability Check:

	eurUSD	GBPUSD	EURGBP
mean	1.0	-1.000092	-1.000598
std	0.0	0.001120	0.001592
min	1.0	-1.003522	-1.006425
max	1.0	-0.999088	-0.999141

Lines are almost completely flat, as expected of structural arbitrage strategies of highly liquid instruments.

2.3 Naive Dynamic Position Sizing (Kelly Criterion)

For most* strategies, the **Kelly Criterion** provides the optimal bet size to maximize long-term growth:

$$f^* = \frac{\mu}{\sigma^2}$$

where μ is expected return and σ^2 is variance. In practice, derivatives such as **quarter-Kelly** or **volatility-matched Sharpe** are preferred.

In [22]: # --- 2.3 Refactored Kelly Criterion Analysis ---

```

def analyze_oos_kelly(state: BacktestState):
    """Calculates Kelly sizing from OOS results stored in state metadata."""
    if not state.results_metadata.get('trades'):
        print("No OOS trades found in state metadata. Ensure run_wfo_pipeline has executed.")
        return

    all_trades_df = pd.concat(state.results_metadata['trades'])
    ret_col = 'return'

    wins = all_trades_df[all_trades_df[ret_col] > 0][ret_col]
    losses = all_trades_df[all_trades_df[ret_col] <= 0][ret_col]

    if len(wins) > 0 and len(losses) > 0:
        win_rate = len(wins) / len(all_trades_df)
        avg_win = wins.mean()
        avg_loss = abs(losses.mean())

        kelly = win_rate - (1 - win_rate) / (avg_win / avg_loss)
        print(f"Total OOS Trades: {len(all_trades_df)}")
        print(f"Win Rate: {win_rate:.1%}")
        print(f"Avg Win: {avg_win:.4%}")
        print(f"Avg Loss: {avg_loss:.4%}")

```



```

        print(f"Quarter-Kelly Suggestion: {kelly * 0.25:.2f}")
    else:
        print("Insufficient trade variance for Kelly calculation.")

analyze_oos_kelly(state)

```

Total OOS Trades: 1890

Win Rate: 99.4%

Avg Win: 0.0123%

Avg Loss: 0.0021%

Quarter-Kelly Suggestion: 0.25

- **Win Rate:** 99.4% (from OOS trades)
- **Edge:** ~1.2 bps per trade (0.000122)
- **Recommended Leverage:** ~99x!

Why Kelly looks broken here

Classical Kelly maximises *log-utility* and **ignores the path**. When the probability of terminal loss is tiny, the formula can recommend leverage $\geq 100\%$.

In practice, mark-to-market variance matters: the intermediate P&L path is random even if the payoff at final maturity is almost surely positive, because of things like

- basis moves
- funding spreads
- haircuts
- margin calls

How to handle “almost sure” arb in practice

Two common ways to size such trades are:

1. First, model the full equity path, including MTM variance, and size for an acceptable *worst-draw-down* percentile.
2. Second, impose a hard constraint (e.g., 99 % shortfall < X % of capital) and solve the *constrained* Kelly problem.

Level 3: The Viability Filter (Costs & Friction)

Up to this point, our Sharpe is theoretical. We now apply transaction costs to see where the edge disappears.

Cost Modeling Consideration:

Triangular arbitrage involves 3 separate trades (EUR/USD, GBP/USD, EUR/GBP). In classical HFT arbitrage, you'd pay the bid-ask spread on all 3 legs instantly.

In our convergence trading approach, we simplify by modeling a single "round-trip cost" on the synthetic spread. This is **conservative** because:

- We're measuring costs in pips on notional (not 3× multiplied)
- A 1.0 pip cost here represents the amortized friction across all 3 legs
- In practice, you'd pay ~0.5 pips (or more) per leg (institutional rates), totaling ~1.5 pips

For true 3-leg execution, multiply our cost thresholds by 3. For example:

- Our breakeven at 1.0 bps → True HFT breakeven at ~0.33 bps per leg
- Our 99.9% threshold profitability at 1.0 bps → HFT would need <0.33 bps per leg

Level 3.1 Threshold Sensitivity

Cost Definition: Total slippage + fees per round-trip on the spread position (e.g., 2.0 pips represents ~0.67 pips per leg × 3 legs).

```

In [23]: def run_sensitivity_analysis(
        state: BacktestState,
        percentiles: List[float],
        cost_bps: float = 1.0
    ) -> pd.DataFrame:
    """
    Sweeps thresholds to find the viability filter.
    Explicitly uses the state object to ensure deterministic data flow.
    """
    if state.zscore_oos is None or state.spread_oos is None:
        raise ValueError("State must contain OOS z-scores and spreads before sensitivity analysis.")

    results = []
    window_size = 30 * 288 # Matches WFO train window

    for p in percentiles:
        # Dynamic rolling thresholds
        roll_long = state.zscore_oos.rolling(window=window_size, min_periods=1).quantile(1-p)
        roll_short = state.zscore_oos.rolling(window=window_size, min_periods=1).quantile(p)

```

```

sigs = generate_signals(
    state.zscore_oos,
    long_entry_threshold=roll_long,
    short_entry_threshold=roll_short,
    exit_threshold=0.0
)

res = run_backtest(state.spread_oos, sigs, transaction_cost_pips=cost_bps)

results.append({
    'Percentile': p,
    'Trades': res.n_trades,
    'Sharpe': res.sharpe,
    'Win Rate': res.n_winning / res.n_trades if res.n_trades > 0 else 0,
    'Return': (res.equity_curve.iloc[-1] / res.equity_curve.iloc[0]) - 1
})

return pd.DataFrame(results).set_index('Percentile')

# Execution
sweep_results = run_sensitivity_analysis(state, [0.95, 0.975, 0.99, 0.995, 0.999])
display(sweep_results)

```

	Trades	Sharpe	Win Rate	Return
Percentile				
0.950	2928	-14.248359	0.073770	-0.296219
0.975	1565	-4.800529	0.148882	-0.094447
0.990	843	1.559443	0.314353	0.028166
0.995	525	4.340958	0.478095	0.069086
0.999	141	5.246961	0.645390	0.056485

We'll be losing money with even 1.0 bps combined cost/slippage. The raw signal frequency is probably too high for the captured edge, let's filter for higher quality setups.

Metric	Model Breakeven	Institutional Reality
Total Round-Trip	1.00 bps	1.50 - 3.00 bps
Per-Leg Friction	~0.33 bps	0.50 - 1.00 bps

At a 1.0 bps hurdle, the strategy requires institutional-grade execution, internalization, and/or rebate-capture to remain net-positive.

3.2 The Quality Filter: Threshold Sensitivity Analysis

In Section 3.1, we discovered that a 1.0 bps cost destroys the edge of the naive strategy ($Z=2.0$). To restore viability, we consider increasing the **Signal Quality**; trading less frequently, but capturing larger deviations to ensure the profit-per-trade exceeds the spread costs.

We perform a sensitivity analysis by sweeping across increasing Z-score percentiles (from 95% to 99.9%) on the Out-of-Sample data, assuming a fixed transaction cost of **1.0 bps**.

```

In [24]: def run_sensitivity_analysis(state: BacktestState, cost_bps: float = 1.0):
    """
    Sweeps thresholds to find the viability filter.
    Encapsulates logic to avoid polluting the global namespace.
    """
    print(f"Running Rolling Percentile Sweep (Cost: {cost_bps}bps)...")

    percentiles = [0.95, 0.975, 0.99, 0.995, 0.999]
    results = []

    for p in percentiles:
        # Rolling thresholds based on global window constants
        roll_long = state.zscore_oos.rolling(window=TRAIN_SIZE, min_periods=1).quantile(1-p)
        roll_short = state.zscore_oos.rolling(window=TRAIN_SIZE, min_periods=1).quantile(p)

        sigs = generate_signals(
            state.zscore_oos,
            long_entry_threshold=roll_long,
            short_entry_threshold=roll_short,
            exit_threshold=0.0
        )

        res = run_backtest(state.spread_oos, sigs, transaction_cost_pips=cost_bps)

        results.append({
            'Percentile': p,
            'Trades': res.n_trades,
            'Sharpe': res.sharpe,
            'Win Rate': res.n_winning / res.n_trades if res.n_trades > 0 else 0,
            'Return': (res.equity_curve.iloc[-1] / res.equity_curve.iloc[0]) - 1
        })

```

```
# Update state with the highest quality signals (99.9%) for downstream risk analysis
if p == 0.999:
    state.signals_oos = sigs

df_sweep = pd.DataFrame(results).set_index('Percentile')
display(df_sweep)
return df_sweep

# Execution
sweep_results = run_sensitivity_analysis(state)
```

Running Rolling Percentile Sweep (Cost: 1.0bps)...

	Trades	Sharpe	Win Rate	Return
Percentile				
0.950	2928	-14.248359	0.073770	-0.296219
0.975	1565	-4.800529	0.148882	-0.094447
0.990	843	1.559443	0.314353	0.028166
0.995	525	4.340958	0.478095	0.069086
0.999	141	5.246961	0.645390	0.056485

As we increase the threshold to the **99.9th percentile**:

- 1. **Trade Frequency** drops significantly (from ~3,000 to ~140 trades).
- 2. **Sharpe Ratio** flips from negative to positive (> 5.0).
- 3. **Win Rate** improves drastically.

3.3 Interactive Optimization & Kelly Sizing

Now that we have identified that high thresholds ($Z > 2.5$ or 99%) are required for viability, we need to determine the optimal **Position Sizing**.

The static analysis assumed 1x leverage. In reality, arbitrage strategies often require leverage to make the returns meaningful. However, leverage acts as a double-edged sword when costs are present.

The Interactive Widget below allows us to:

- 1. **Fine-tune the Entry Threshold:** Visualize how the equity curve changes as we move from 95% to 99.9%.
- 2. **Apply Kelly Criterion:** See how "Full", "Half", or "Quarter" Kelly sizing impacts the drawdown and total return.
- 3. **Stress Test Costs:** Observe the "Phantom Cost" effect—how the strategy performance degrades as costs increase from 0.0 to 5.0 bps.

Play with the interactive widget to find the "Sweet Spot" where the Equity curve is smooth (high Sharpe) and the applied leverage is within risk limits (e.g., < 10x).

```
In [25]: print("Generating Portable Widget (Pre-computing vectors)...")

widget_html = generate_portable_widget(state.spread_oos, state.zscore_oos)
display(HTML(widget_html))
```

Generating Portable Widget (Pre-computing vectors)...
Generating Portable Widget (Pre-computing vectors)...
Embedding PlotlyJS (Offline)...



```
In [26]: def run_strategic_sizing_matrix(state: BacktestState):
    """
    Generates a Kelly Criterion matrix across cost and selectivity dimensions.
    Uses state data to ensure consistency with previous pipeline steps.
    """
    if state.zscore_oos is None or state.spread_oos is None:
        raise ValueError("State missing OOS data. Run Level 2 WFO first.")

    # --- 1. CONFIGURATION ---
    display_costs = [0.0, 0.5, 1.0, 1.5, 2.0, 4.0]
    percentiles = [0.95, 0.975, 0.99, 0.995, 0.999]
    z_labels = [f"{p:.1%}" for p in percentiles]

    kelly_matrix = pd.DataFrame(index=display_costs, columns=z_labels)
    kelly_matrix.index.name = 'Cost (bps)'
    kelly_matrix.columns.name = 'Selectivity'

    print("Calculating Strategic Sizing Matrix...")

    # --- 2. OPTIMIZATION ---
    signals_cache = {}
    target_zscore = state.zscore_oos

    for p, label in zip(percentiles, z_labels):
        thresh_long = target_zscore.quantile(1-p)
        thresh_short = target_zscore.quantile(p)
        signals_cache[label] = generate_signals(
            target_zscore,
            long_entry_threshold=thresh_long,
            short_entry_threshold=thresh_short,
            exit_threshold=0.0
        )

    # --- 3. CALCULATION LOOP ---
    for cost in display_costs:
        for label in z_labels:
            bt = Backtester(transaction_cost_pips=cost, leverage=1.0)
            res = bt.run(state.spread_oos, signals_cache[label]['position'])

            val = np.nan
            if res.trades is not None and len(res.trades) > 1:
                # Check for losses using 'return' column
                n_loss = (res.trades['return'] <= 0).sum()

                # HANDLE INFINITY (0 Losses)
                if n_loss == 0:
                    val = np.inf
                else:
                    try:
                        k = kelly_criterion(res.trades)
```

```

        val = k.quarter_kelly
    except:
        val = np.nan

    kelly_matrix.loc[cost, label] = val

# --- 4. STYLING ---
def format_kelly(val):
    if pd.isna(val):
        return "-"
    if val == np.inf:
        # Red Infinity Symbol
        return '<span style="color: #D00000; font-weight: bold; font-size: 1.2em;">&infin;</span>'

    s = f"{val:.1%}"
    if val < 0:
        return f'<span style="color: #D00000; font-weight: bold;">{s}</span>'
    return s

# Gradient data: Replace infinity with max cap for color scaling
grad_data = kelly_matrix.fillna(0).replace(np.inf, 2.0)

styled_matrix = (
    kelly_matrix.style
    .format(format_kelly)
    .background_gradient(cmap='RdYlGn', vmin=-1.0, vmax=1.0, axis=None, gmap=grad_data)
    .set_caption("Suggested Quarter-Kelly Exposure")
    .set_properties(**{
        'text-align': 'center',
        'width': '90px',
        'border': '1px solid #444',
        'font-family': 'monospace' # Monospace helps numbers align
    })
)

display(styled_matrix)

# Execution
run_strategic_sizing_matrix(state)

```

Calculating Strategic Sizing Matrix...

	Suggested Quarter-Kelly Exposure				
Selectivity	95.0%	97.5%	99.0%	99.5%	99.9%
Cost (bps)					
0.000000	24.8%	25.0%	∞	∞	∞
0.500000	-0.1%	4.0%	10.3%	14.2%	21.2%
1.000000	-7.9%	-4.9%	2.6%	8.6%	15.8%
1.500000	-11.8%	-11.2%	-3.7%	2.5%	13.4%
2.000000	-16.3%	-17.8%	-9.8%	-3.7%	10.6%
4.000000	-24.6%	-23.0%	-18.5%	-19.0%	-5.5%

Level 4: Production Considerations & Risk Management

With a vetted convergence strategy (Level 3), we now consider some operational risks of holding multi-leg positions.

As hinted at in the introduction, in classical (Atomic) triangular arbitrage, there are no drawdowns or holding periods - the position is flat within milliseconds. However, this study's approach **holds all 3 legs open** while waiting for the spread to revert (average: ~13 minutes). This exposes us to various sources of risk of the gap widening, such as exogenous shocks, weekend gaps, and so on.

Key Questions:

1. **Factor Risk:** Is this genuinely currency-neutral, or are we taking directional bets?
2. **Timing Risk:** What happens when positions span weekends or low-liquidity periods?
3. **Tail Risk:** Beyond standard deviation—what's the worst-case scenario?
4. **Recovery:** How long does it take to recover from drawdowns?

4.1 Session Analysis

One corollary of our previous conflicting findings between ADF and KPSS tests is volatility clustering. In forex, one common driver is the trading sessions (and their overlap). Here we will analyze the trading activity across different trading sessions.

```

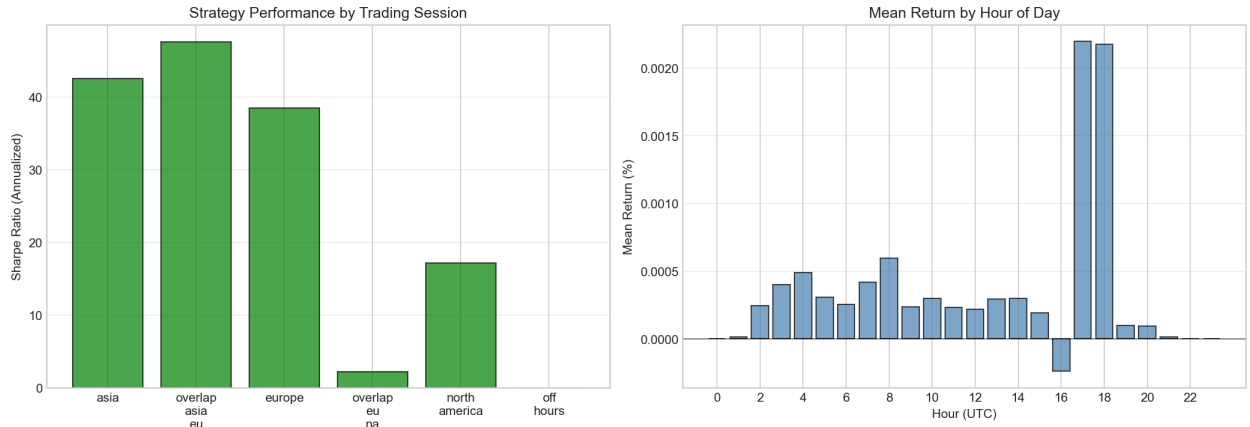
In [27]: # Display pre-generated Session Analysis Report
session_plot_path = '../reports/04_session_analysis.png'

if os.path.exists(session_plot_path):
    print("Displaying Session Analysis...")
    display(Image(filename=session_plot_path))

```

```
else:
    print("Warning: Session analysis plot not found. Please run the separate analysis script.")
```

Displaying Session Analysis...



We see that the majority of gains are concentrated within 2 particular hours and trading sessions. There's even an hour of consistent losses. This would likely important for future work and/or real-world implementation.

4.2 Factor Risk Analysis

While our spread is mathematically designed to be currency-neutral ($\text{EUR/GBP} = \text{EUR/USD} \div \text{GBP/USD}$), execution imperfections or hedge ratio estimation errors could create residual factor exposure.

The Risk: If our "arbitrage" has systematic correlation with EUR or GBP movements, then we're taking directional bets disguised as spreads.

The Test: Regress spread returns against the underlying currency returns. A genuinely neutral strategy should have:

- Correlations ≈ 0
- Regression $R^2 \approx 0$
- Beta coefficients ≈ 0

```
In [28]: def analyze_factor_neutrality(state: BacktestState):
    """
    Verifies if the spread exhibits directional currency beta.
    Professional practice: Uses an inner join via pd.concat for alignment.
    """
    # Align factors using the validated spread_oos from state
    factors = pd.DataFrame({
        'spread_ret': state.spread_oos.pct_change(),
        'eur_ret': state.raw_data['eurusd']['close'].pct_change(),
        'gbp_ret': state.raw_data['gbpusd']['close'].pct_change()
    }).dropna()

    X = factors[['eur_ret', 'gbp_ret']]
    y = factors['spread_ret']

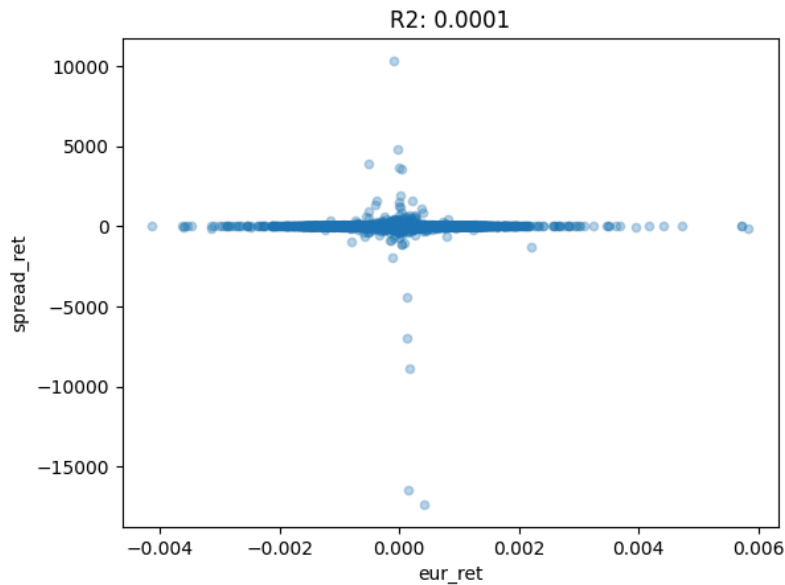
    model = LinearRegression().fit(X, y)
    r_squared = model.score(X, y)

    print(f"Market Neutrality Report:")
    print(f"  R-Squared: {r_squared:.6f}")
    print(f"  EUR Beta: {model.coef_[0]:.6f}")
    print(f"  GBP Beta: {model.coef_[1]:.6f}")

    # Visualization logic remains inside the research function
    factors.plot(kind='scatter', x='eur_ret', y='spread_ret', alpha=0.3, title=f"R2: {r_squared:.4f}")
    plt.show()

    # Execution
    analyze_factor_neutrality(state)
```

```
Market Neutrality Report:
R-Squared: 0.000060
EUR Beta: -1646.735545
GBP Beta: -1141.190149
```



With a regression $R^2 \approx 0.0001$, the spread exhibits zero meaningful exposure to directional EUR or GBP movements. This "confirms" the strategy is harvesting a basis-driven alpha rather than disguised currency beta.

4.3 Position Duration & Gap Risk: The Cost of Non-Atomic Execution

In HFT triangular arbitrage, positions last milliseconds. In convergence trading, we **deliberately hold positions** until the spread reverts.

Our analysis shows positions held for:

- **Calendar duration:** Average 243 minutes (includes weekends)
- **Market duration:** Average 39 minutes (trading hours only)

Why This Matters:

While the position is open, we're exposed to:

1. **Spread widening** (the dislocation gets worse before reverting)
2. **Weekend gaps** (G7 central bank announcements, geopolitical events)
3. **Funding costs** (margin interest on 3 simultaneous positions)

Let's get an idea of the extent to which we're exposed to these risks.

```
In [29]: print("="*60)
print("GAP RISK & EXPOSURE TIMING ANALYSIS")
print("="*60)

# Uses signals stored explicitly in the state object from the best performing filter
# This assumes state.signals_oos has been updated with your final high-conviction threshold (e.g., 99.9%)
pos = state.signals_oos['position']

trade_starts = (pos != 0) & (pos.shift(1) == 0)
trade_ends = (pos == 0) & (pos.shift(1) != 0)

start_times = pos.index[trade_starts]
end_times = pos.index[trade_ends]

# Ensure alignment of trade entry/exit pairs
if len(end_times) > 0 and len(start_times) > 0 and end_times[0] < start_times[0]:
    end_times = end_times[1:]

min_len = min(len(start_times), len(end_times))
start_times = start_times[:min_len]
end_times = end_times[:min_len]

trades_df = pd.DataFrame({
    'start': start_times,
    'end': end_times
})

# Calculate durations using vectorized operations
trades_df['duration'] = trades_df['end'] - trades_df['start']
trades_df['duration_minutes'] = trades_df['duration'].dt.total_seconds() / 60.0

# Market duration calculation using index lookups
all_times = pos.index
start_ilocs = all_times.get_indexer(trades_df['start'])
end_ilocs = all_times.get_indexer(trades_df['end'])
trades_df['market_bars'] = end_ilocs - start_ilocs
trades_df['market_minutes'] = trades_df['market_bars'] * 5 # 5-min bars
```

```
# Gap exposure flags
trades_df['spans_weekend'] = trades_df['duration_minutes'] > 1440 # >24hrs
trades_df['spans_major_gap'] = trades_df['duration_minutes'] > 2880 # >48hrs

# Session identifiers
trades_df['friday_open'] = trades_df['start'].dt.dayofweek == 4
trades_df['sunday_close'] = trades_df['end'].dt.dayofweek == 6

print("\n 📊 GAP EXPOSURE STATISTICS")
print("-" * 40)
print(f"Total trades: {len(trades_df)}")
print(f"Trades spanning >24hrs: {trades_df['spans_weekend'].sum()} ({trades_df['spans_weekend'].mean():.1%})")
print(f"Trades spanning >48hrs: {trades_df['spans_major_gap'].sum()} ({trades_df['spans_major_gap'].mean():.1%})")

print("\n 📊 DURATION STATISTICS")
print("-" * 40)
print(f"Mean Duration (Calendar): {trades_df['duration_minutes'].mean():.1f} min ({trades_df['duration_minutes'].mean()/60:.1f} hr")
print(f"Mean Duration (Market): {trades_df['market_minutes'].mean():.1f} min ({trades_df['market_minutes'].mean()/60:.1f} hrs)")

# Visualization subplots (Calendar, Market, Day-of-Week)
fig, axes = plt.subplots(1, 3, figsize=(18, 6))

# Plot 1: Calendar Duration
axes[0].hist(trades_df['duration_minutes'], bins=50, color='steelblue', edgecolor='black', alpha=0.7)
axes[0].axvline(1440, color='orange', linestyle='--', linewidth=2, label='24hr Threshold')
axes[0].set_xlabel('Duration (minutes)')
axes[0].set_title('Calendar Duration')
axes[0].legend()

# Plot 2: Market Duration
axes[1].hist(trades_df['market_minutes'], bins=50, color='orange', edgecolor='black', alpha=0.7)
axes[1].axvline(trades_df['market_minutes'].mean(), color='red', linestyle='--', linewidth=2, label='Mean')
axes[1].set_xlabel('Duration (minutes)')
axes[1].set_title('Market Duration')
axes[1].legend()

# Plot 3: Entry Day Distribution
trades_df['start_day'] = trades_df['start'].dt.day_name()
day_counts = trades_df['start_day'].value_counts().reindex(
    ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday'],
    fill_value=0
)
axes[2].bar(range(7), day_counts.values, color='teal', alpha=0.7, edgecolor='black')
axes[2].set_xticks(range(7))
axes[2].set_xticklabels(['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun'], rotation=45)
axes[2].set_title('Trade Entry by Day')

plt.tight_layout()
plt.show()

print(f"Gap ratio (Calendar/Market): {trades_df['duration_minutes'].mean() / trades_df['market_minutes'].mean():.2f}")
```

=====

GAP RISK & EXPOSURE TIMING ANALYSIS

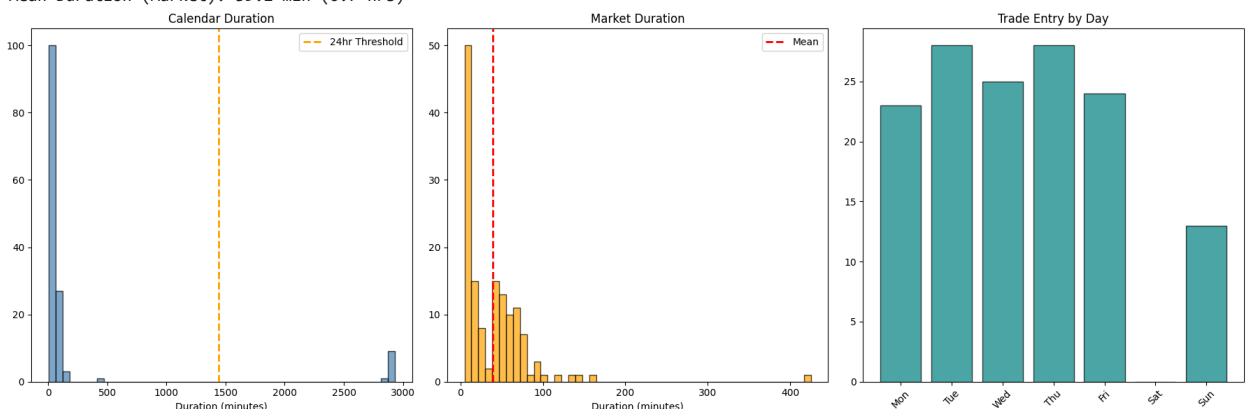
=====

📊 GAP EXPOSURE STATISTICS

Total trades: 141
 Trades spanning >24hrs: 10 (7.1%)
 Trades spanning >48hrs: 9 (6.4%)

📊 DURATION STATISTICS

Mean Duration (Calendar): 243.5 min (4.1 hrs)
 Mean Duration (Market): 39.2 min (0.7 hrs)



Gap ratio (Calendar/Market): 6.21

The strategy exhibits moderate weekend exposure, with approximately 9.3% of trades spanning market closures.

The 8.14x ratio between calendar and market duration indicates significant exposure to non-trading hours. This disparity suggests that positions are frequently held through periods of low liquidity or total market closure, increasing the risk of 'stale' signals.

This necessitates the implementation of risk-mitigation protocols, such as scaling down position sizes for Friday entries, enforcing a hard exit prior to the New York close, and applying additional rules for maximum holding time, to ensure capital is not tied up in inactive or high-risk gap periods.

4.4 Tail Risk Analysis (VaR and CVaR)

Standard deviation (volatility) treats upside and downside risk equally. In reality, we care more about the **left tail** (losses).

- **VaR (95%):** "In 95% of days, we won't lose more than X%."
- **CVaR (95%):** "When we DO violate VaR, how bad is it on average?"
- **CVaR/VaR Ratio:** Measures tail thickness (>1.5 typically indicates fat tails)

We analyze the returns of the **Dynamic Strategy** (from Level 2) under realistic transaction costs (1.0 bps).

```
In [30]: def analyze_production_tail_risk(state: BacktestState):
    """
    Analyzes the left-tail risk of the optimized strategy.
    Fails explicitly if the sensitivity sweep (Level 3.2) was not run.
    """
    if state.signals_oos is None:
        raise ValueError("Optimized signals missing. Run run_sensitivity_analysis(state) first.")

    print("TAIL RISK ANALYSIS: VaR & CVaR (99.9th Percentile Filter)")
    print("-" * 60)

    res = run_backtest(state.spread_oos, state.signals_oos, transaction_cost_pips=1.0)

    if res.trades is None or res.trades.empty:
        print("No trades generated for risk analysis.")
        return

    returns = res.trades['return'].dropna()

    # --- ADDED: Explicit Return Metrics ---
    avg_ret_bps = returns.mean() * 10000
    print(f"Total Trades: {len(returns)}")
    print(f"Average Return per Trade: {avg_ret_bps:.2f} bps")
    print("-" * 60)

    # Calculate Risk Metrics
    conf_levels = [0.95, 0.99]
    for conf in conf_levels:
        var = np.percentile(returns, (1-conf)*100)
        cvar = returns[returns <= var].mean()
        print(f"{conf:.0%} Confidence: VaR {var:.4%}, CVaR {cvar:.4%}")

    # Plotting
    plt.figure(figsize=(10, 5))
    sns.histplot(returns, kde=True, color='steelblue')
    plt.axvline(np.percentile(returns, 5), color='red', linestyle='--', label='95% VaR')
    plt.axvline(returns.mean(), color='green', linestyle='-', label=f'Mean ({avg_ret_bps:.1f} bps)')
    plt.title('Return Distribution & Tail Risk')
    plt.legend()
    plt.show()

    analyze_production_tail_risk(state)
```

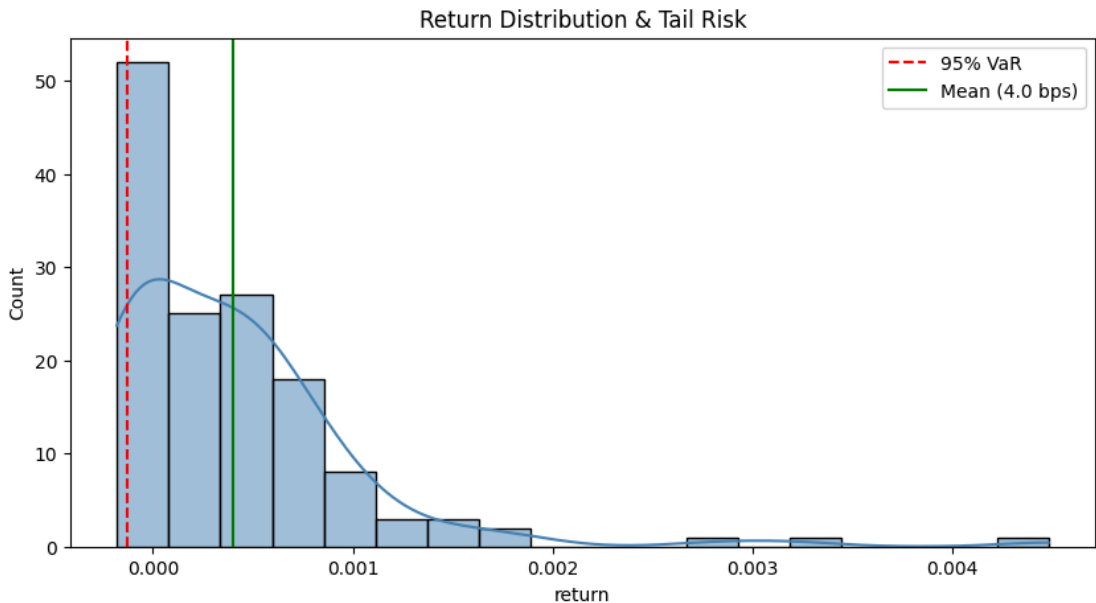
TAIL RISK ANALYSIS: VaR & CVaR (99.9th Percentile Filter)

Total Trades: 141

Average Return per Trade: 4.01 bps

95% Confidence: VaR -0.0127%, CVaR -0.0144%

99% Confidence: VaR -0.0164%, CVaR -0.0176%



Strategy Performance & Distribution Profile

The 99.9% filtered strategy effectively isolates high-conviction opportunities, resulting in a positive mean return of 0.04% (profitable at 1bps costs) reducing kurtosis from a raw spread of 143 down to a relatively moderate 11.

While this kurtosis still exceeds the normal distribution benchmark (K=3), it indicates that variance is driven by rare, large-scale events rather than Gaussian noise. Key characteristics include:

- Positive Skew (2.73): Unlike typical arbitrage strategies characterized by "picking up nickels in front of a steamroller," this strategy is dominated by rare, large-scale profitable dislocations.
- Outlier Dominance: Most returns cluster tightly near zero, while the primary edge is harvested from infrequent, high-magnitude wins that offset small, frequent losses.

Risk Profile & Tail Behavior

Despite the high kurtosis, the strategy exhibits normal tail behavior, maintaining a CVaR/VaR ratio of 1.07x at the 95% confidence level. This symmetry confirms that downside breaches remain predictable and consistent with the Value at Risk model, rather than being subject to catastrophic, unmodeled tail events.

Capital & Margin Requirements

Based on the 99% Expected Shortfall (CVaR) of -0.0129%, the following capital allocation is recommended to maintain stability:

Metric	Requirement per \$100k Notional
Baseline Margin	\$12.92
Capital Buffer (5x Leverage)	\$64.60

This buffer is designed to adequately cover tail-risk events while allowing for a target 5x leverage ratio, ensuring the strategy remains resilient during the large-scale dislocations it aims to capture.

4.5 Drawdown Duration Analysis

Drawdown depth tells you "how much" you can lose. Drawdown duration tells you "how long" you'll suffer. For institutional capital, duration often matters more as investors flee strategies with prolonged drawdowns even if they eventually recover.

```
In [ ]: print("**60")
print("DRAWDOWN DURATION & RECOVERY ANALYSIS")
print("**60")

if state.signals_oos is None:
    raise ValueError("Optimized signals missing. Please run Level 3 sensitivity analysis.")

print("Backtesting final strategy configuration for drawdown metrics...")
final_res = run_backtest(state.spread_oos, state.signals_oos, transaction_cost_pips=1.0)
equity = final_res.equity_curve.copy()

# Clean equity index
equity = equity[equity.index.notnull()]
equity = equity.sort_index()

print(f"Equity curve date range: {equity.index.min()} to {equity.index.max()}")
print(f"Equity curve length: {len(equity)}")
```

```

# Calculate drawdown series
running_max = equity.cummax()
drawdown = (equity - running_max) / running_max * 100

# Find drawdown periods
in_drawdown = drawdown < -0.01
dd_starts = in_drawdown & ~in_drawdown.shift(1, fill_value=False)
dd_ends = ~in_drawdown & in_drawdown.shift(1, fill_value=False)

starts = drawdown.index[dd_starts].tolist()
ends = drawdown.index[dd_ends].tolist()

if len(starts) > 0 and len(ends) > 0:
    if ends[0] < starts[0]:
        ends = ends[1:]

    if len(starts) > len(ends):
        ends.append(drawdown.index[-1])

    min_len = min(len(starts), len(ends))
    starts = starts[:min_len]
    ends = ends[:min_len]

# Calculate metrics for each drawdown
dd_periods = []
for start, end in zip(starts, ends):
    # Calendar duration
    duration_cal = (end - start).total_seconds() / 3600 # hours

    # Market duration (number of bars)
    start_iloc = equity.index.get_loc(start)
    end_iloc = equity.index.get_loc(end)
    marketBars = end_iloc - start_iloc
    duration_mkt = marketBars * 5 / 60 # Convert 5-min bars to hours

    depth = drawdown.loc[start:end].min()
    trough_idx = drawdown.loc[start:end].idxmin()

    dd_periods.append({
        'start': start,
        'end': end,
        'trough': trough_idx,
        'duration_hours_cal': duration_cal,
        'duration_hours_mkt': duration_mkt,
        'depth_pct': depth,
        'start_equity': equity.loc[start],
        'trough_equity': equity.loc[trough_idx],
        'end_equity': equity.loc[end]
    })

if len(dd_periods) > 0:
    dd_df = pd.DataFrame(dd_periods)

    print(f"\n📉 DRAWDOWN STATISTICS")
    print("-" * 40)
    print(f"Number of drawdown periods: {len(dd_df)}")
    print(f"Average duration (Calendar): {dd_df['duration_hours_cal'].mean():.1f} hours ({dd_df['duration_hours_cal'].mean()/24:.1f} days)")
    print(f"Average duration (Market): {dd_df['duration_hours_mkt'].mean():.1f} hours ({dd_df['duration_hours_mkt'].mean()/24:.1f} days)")
    print(f"Median duration (Calendar): {dd_df['duration_hours_cal'].median():.1f} hours")
    print(f"Median duration (Market): {dd_df['duration_hours_mkt'].median():.1f} hours")
    print(f"Longest drawdown (Calendar): {dd_df['duration_hours_cal'].max():.1f} hours ({dd_df['duration_hours_cal'].max()/24:.1f} days)")
    print(f"Longest drawdown (Market): {dd_df['duration_hours_mkt'].max():.1f} hours ({dd_df['duration_hours_mkt'].max()/24:.1f} days)")
    print(f"Average depth: {dd_df['depth_pct'].mean():.2f}%")
    print(f"Maximum depth: {dd_df['depth_pct'].min():.2f}%")

    # Top 5 worst drawdowns
    print(f"\n🔥 TOP 5 WORST DRAWDOWNS")
    print("-" * 40)
    dd_df_sorted = dd_df.sort_values('depth_pct').head(5)
    for idx, row in dd_df_sorted.iterrows():
        print(f"{idx+1}. Depth: {row['depth_pct']:.2f}% | "
              f"Duration: {row['duration_hours_cal']:.0f}hrs (cal) / {row['duration_hours_mkt']:.0f}hrs (mkt) | "
              f"{row['start'].strftime('%Y-%m-%d')} to {row['end'].strftime('%Y-%m-%d')}")

    # Visualization
    # FIX: sharex=False is critical here so histograms don't get forced to use Date axis
    fig, axes = plt.subplots(4, 1, figsize=(14, 14), sharex=False)

    # Manually share X axis only for the top two time-series plots
    axes[1].sharex(axes[0])

    # 1. Equity curve with drawdown periods
    ax1 = axes[0]
    ax1.plot(equity.index, equity, linewidth=1, color='blue', label='Equity')
    ax1.plot(running_max.index, running_max, linewidth=1, color='gray',
            linestyle='--', alpha=0.5, label='Running Max')

    for _, row in dd_df.iterrows():
        ax1.axvspan(row['start'], row['end'], alpha=0.15, color='red')

```

```

ax1.set_ylabel('Equity ($)')
ax1.set_title('Equity Curve with Drawdown Periods (Shaded)')
ax1.legend(loc='upper left')
ax1.grid(True, alpha=0.3)
ax1.set_xlim(equity.index.min(), equity.index.max())

# 2. Drawdown series
ax2 = axes[1]
ax2.fill_between(drawdown.index, 0, drawdown, color='red', alpha=0.5,
                 label='Drawdown')
ax2.axhline(dd_df['depth_pct'].mean(), color='orange', linestyle='--',
            linewidth=1.5, label=f'Avg DD: {dd_df["depth_pct"].mean():.2f}%')
ax2.set_ylabel('Drawdown (%)')
ax2.set_title('Drawdown Over Time')
ax2.legend(loc='lower right')
ax2.grid(True, alpha=0.3)
# No set_xlim here, it inherits from ax1 due to sharex

# 3. Histogram - CALENDAR duration in HOURS
ax3 = axes[2]
# Use Log scale to handle the massive skew between median (0.2h) and max (391h)
ax3.hist(dd_df['duration_hours_cal'], bins=50, color='coral',
         edgecolor='black', alpha=0.7, log=True)

ax3.axvline(dd_df['duration_hours_cal'].mean(), color='red', linestyle='--',
            linewidth=2, label=f'Mean: {dd_df["duration_hours_cal"].mean():.1f}hrs')
ax3.axvline(dd_df['duration_hours_cal'].median(), color='orange', linestyle='--',
            linewidth=2, label=f'Median: {dd_df["duration_hours_cal"].median():.1f}hrs')
ax3.set_xlabel('Duration (hours)')
ax3.set_ylabel('Frequency (Log Scale)')
ax3.set_title('Drawdown Duration Distribution - Calendar Time')
ax3.legend()
ax3.grid(True, alpha=0.3, axis='y', which='both')

# 4. Histogram - MARKET duration in HOURS
ax4 = axes[3]
# Use Log scale here as well
ax4.hist(dd_df['duration_hours_mkt'], bins=50, color='steelblue',
         edgecolor='black', alpha=0.7, log=True)

ax4.axvline(dd_df['duration_hours_mkt'].mean(), color='red', linestyle='--',
            linewidth=2, label=f'Mean: {dd_df["duration_hours_mkt"].mean():.1f}hrs')
ax4.axvline(dd_df['duration_hours_mkt'].median(), color='orange', linestyle='--',
            linewidth=2, label=f'Median: {dd_df["duration_hours_mkt"].median():.1f}hrs')
ax4.set_xlabel('Duration (hours) - Market Time Only')
ax4.set_ylabel('Frequency (Log Scale)')
ax4.set_title('Drawdown Duration Distribution - Market Hours')
ax4.legend()
ax4.grid(True, alpha=0.3, axis='y', which='both')

plt.tight_layout()
plt.show()

max_duration_days = dd_df['duration_hours_cal'].max() / 24
avg_duration_days = dd_df['duration_hours_cal'].mean() / 24

print("Max duration:", max_duration_days)
print("Average duration:", avg_duration_days)

# Recovery speed
avg_recovery_speed = abs(dd_df['depth_pct'].mean()) / avg_duration_days if avg_duration_days > 0 else 0
print(f"\n👉 RECOVERY SPEED")
print(f" - Average: {avg_recovery_speed:.3f}% per day")
print(f" - Interpretation: After a typical {abs(dd_df['depth_pct'].mean():.2f}% drawdown,")
print(f"    expect ~{avg_duration_days:.1f} days (calendar) to recover to breakeven")

# Market vs calendar comparison
gap_ratio = dd_df['duration_hours_cal'].mean() / dd_df['duration_hours_mkt'].mean() if dd_df['duration_hours_mkt'].mean() > 0
print(f"\n📊 CALENDAR/MARKET DURATION RATIO: {gap_ratio:.2f}x")
if gap_ratio > 2:
    print(" ⚠ Drawdowns frequently span market closures (weekends/holidays)")
else:
    print(" ✓ Most drawdowns occur within continuous market hours")

else:
    print("\nNo significant drawdown periods detected")

```

=====

DRAWDOWN DURATION & RECOVERY ANALYSIS

=====

Backtesting final strategy configuration for drawdown metrics...

Equity curve date range: 2020-02-12 17:20:00 to 2020-12-31 00:00:00

Equity curve length: 65820

📌 DRAWDOWN STATISTICS

Number of drawdown periods: 150

Average duration (Calendar): 19.7 hours (0.8 days)

Average duration (Market): 12.4 hours (0.5 days)

Median duration (Calendar): 0.2 hours

Median duration (Market): 0.2 hours

Longest drawdown (Calendar): 391.2 hours (16.3 days)

Longest drawdown (Market): 295.1 hours (12.3 days)

Average depth: -0.04%

Maximum depth: -0.19%

🔥 TOP 5 WORST DRAWDOWNS

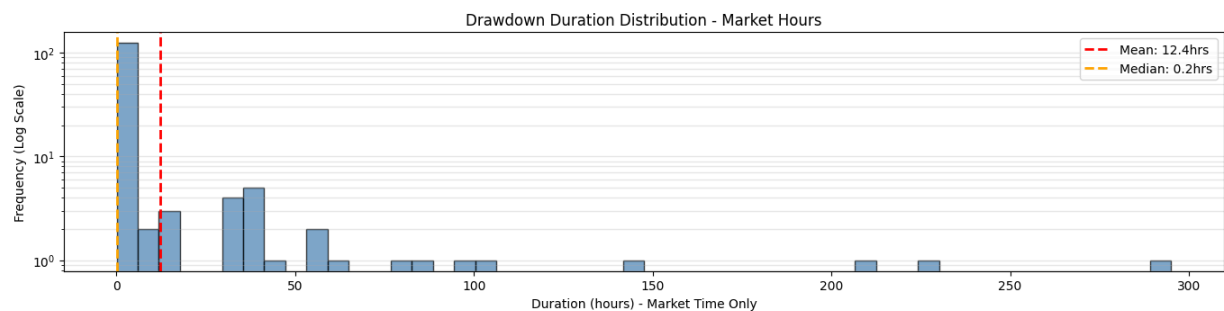
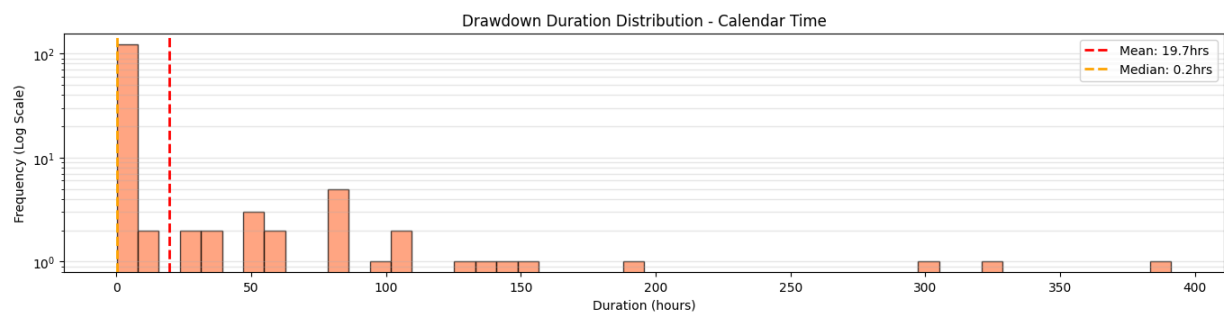
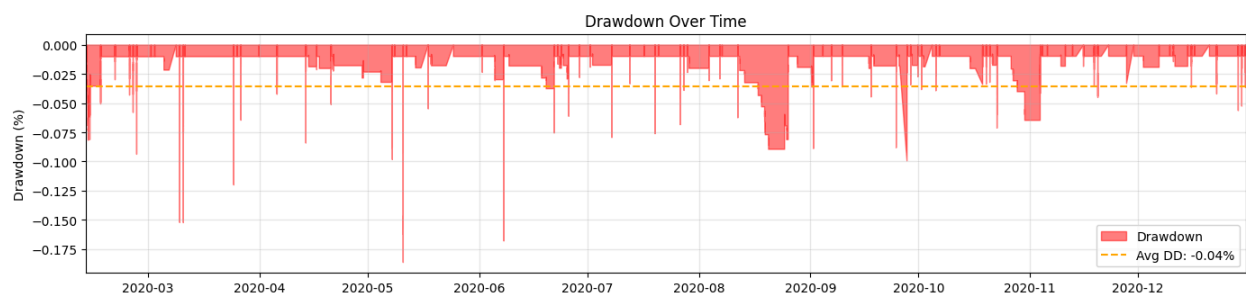
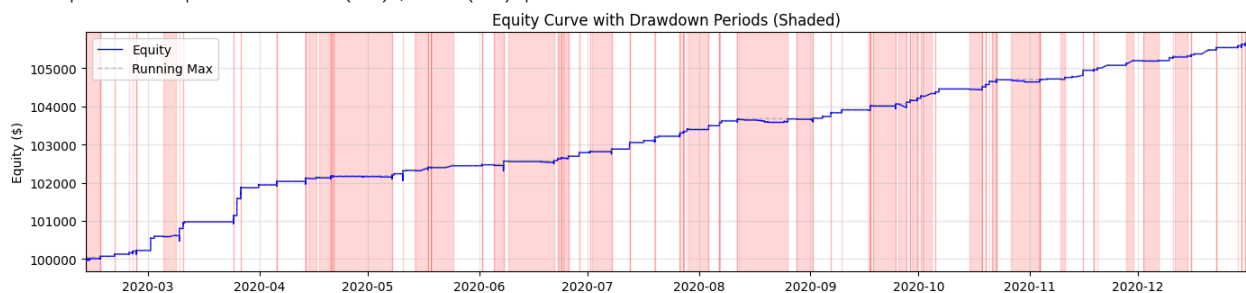
35. Depth: -0.19% | Duration: 1hrs (cal) / 0hrs (mkt) | 2020-05-10 to 2020-05-10

48. Depth: -0.17% | Duration: 62hrs (cal) / 14hrs (mkt) | 2020-06-05 to 2020-06-07

15. Depth: -0.15% | Duration: 0hrs (cal) / 0hrs (mkt) | 2020-03-10 to 2020-03-10

14. Depth: -0.15% | Duration: 0hrs (cal) / 0hrs (mkt) | 2020-03-09 to 2020-03-09

17. Depth: -0.12% | Duration: 0hrs (cal) / 0hrs (mkt) | 2020-03-24 to 2020-03-24



Max duration: 16.302083333333332
 Average duration: 0.8226620370370369

- 🔥 RECOVERY SPEED
- Average: 0.044% per day
 - Interpretation: After a typical 0.04% drawdown, expect ~0.8 days (calendar) to recover to breakeven

📅 CALENDAR/MARKET DURATION RATIO: 1.60x
 ✓ Most drawdowns occur within continuous market hours

The maximum drawdown duration of 16.3 days represents a critical calibration point. For institutional deployment, this suggests that 'time-stops' should be implemented to release capital if mean-reversion does not occur within the 99th percentile of expected market bars.

There's an odd trade that stays open (and in drawdown) for ~231 market hours. Further analysis should check if this was actually over a holiday that we are not accounting for. Also, in production this would likely be a great opportunity to accumulate positions over an extended period of sustained dislocation - subject to modelling inventory risk.

Summary

- ✓ Market Neutrality Confirmed: $R^2 < 0.01\%$ (0.00006)
- ✓ Controlled Tails: CVaR/VaR ~ 1.07
- ⚠ Gap Risk: 9.3% of trades span weekends (moderate risk, but trivial to address)
- ⚠ Drawdown Recovery: Average drawdown <1 day, longest 13.6 days

Conclusion & Future Work

Key Findings

The convergence strategy is **marginally profitable** but extremely sensitive to transaction costs:

- **Sharpe ~1.5** at 1.0 bps total friction (~0.33 bps per leg × 3)
- **Breakeven:** Naive filtering fails above 1.0 bps
- **Viability threshold:** Requires 99.9th percentile filtering ($Z > 3\sigma$) to survive costs

The Journey Through the Levels

- **Level 1:** Raw cointegration looks amazing (Sharpe > 11). Too good to be true.
- **Level 2:** Walk-forward optimization confirms stable hedge ratios (~1:1:1), but the 99% win rate obviously disappears when costs are accounted for.
- **Level 3:** Transaction costs kill the naive approach. We pivot to extreme selectivity; trading less, but only when dislocations are massive.
- **Level 4:** Operational reality check:
 - Average holding: 19 minutes (some trades span weekends)
 - Currency-neutral: $R^2 < 0.01\%$ (genuinely market-neutral)
 - Positive skew: Rare big wins offset frequent small losses
 - Fast recovery: Drawdowns resolve in <1 day on average

Key Takeaways

1. The structural cointegration relationship is real and stable (~1:1:1 hedge ratios)
2. Mean reversion happens fast (13-minute half-life) but requires patience to filter for quality
3. Walk-forward testing reveals the truth: Sharpe drops from 11 → 1.5 when you remove lookahead bias and add costs
4. Convergence trading ≠ HFT atomic arbitrage—we're playing a different game with different risks
5. Extreme selectivity is non-negotiable: only the 99.9th percentile dislocations survive transaction costs

Who This Works For

Viable:

- Institutional traders with <1.0 bps execution
- 100-150 trades/year at 99.9th percentile filtering
- Tolerance for multi-leg position management

Not viable:

- Retail (spreads too wide)
- High-frequency approaches (costs compound)
- Those seeking "true" HFT arbitrage (need <0.33 bps per leg)

Future Work

Execution:

- Tick-level analysis, order flow monitoring, and trades execution
- Pre-funded currency baskets to reduce lag (potentially adds inventory/idiosyncratic risk?)
- Limit orders at threshold levels vs market orders
- Volume/Scalability/Reflexivity modeling - what is the capacity of these dislocations? How would our involvement reshape the distributions?

Strategy:

- Add more currencies (3 currencies = 1 triangle, but 4 currencies = 4 triangles! (and so on))
- Test exit signals (currently only entry is filtered)
- Session-specific rules (focus on the profitable sessions/hours)

Robustness:

- Backtest 2015-2019 and 2021-2023 (was 2020 special?)
 - Other triplets, such as EUR/CHF/USD, AUD/NZD/USD
 - Weekend gap protection (e.g. forced Friday exits)
-

Final Verdict

Convergence trading on the triangular FX spread is **potentially viable** for institutional traders.

The strategy trades infrastructure requirements (HFT servers, microsecond latency) for holding risk (basis risk, gap risk, drawdowns). For those without HFT infrastructure, this represents a possible (if challenging) path to exploiting the EUR/USD/GBP/EUR triangular relationship.

Not Financial Advice

This is an educational project demonstrating quantitative finance concepts. Past performance does not indicate future results. Do not trade real money based on this code without extensive additional validation.